Discrete Choice Models for Credit Card Offers

Amirhossein Javaheri

Feb. 2026

Chapter 1

Literature Review

Discrete choice models represent a fundamental framework in economics, marketing, operations research, and related fields for understanding and predicting how individuals make choices among a finite set of alternatives. These models are essential for analyzing consumer behavior, estimating demand functions, and optimizing product assortments.

A discrete choice model describes how a decision-maker selects one alternative from a finite choice set $S \subseteq \mathcal{S}$. A decision-maker i confronted with a choice set $S_i \subseteq \mathcal{S}$ selects an alternative $j \in S_i$ based on an underlying utility-maximization principle.

There have been significant methodological advances to discrete choice modeling, ranging from classical econometric approaches to modern deep learning techniques. This report reviews the primary methodologies for solving discrete choice problems, categorizing them into foundational random utility maximization (RUM) frameworks, classical econometric specifications, advanced approaches for handling endogeneity and market-level phenomena, and modern neural network-based methods. We emphasize both the theoretical foundations and practical implementation considerations for each approach.

Methodologically, the field spans from classical parametric random utility models (RUM) to modern neural-network-based architectures. This review organizes methods into: (i) foundational RUM and classical logit-type models; (ii) latent class and interaction-augmented MNL variants such as Halo MNL and its low-rank generalization; (iii) neural models including RUMnet, TasteNet, Learning-MNL, and ResLogit; and (iv) related neural and kernel methods for context-dependent choice models.

Throughout, individual-level features (covariates) are denoted by $\mathbf{x}_i \in \mathbb{R}^{d_i}$, alternative features by $\mathbf{x}_j \in \mathbb{R}^{d_x}$ (if fixed for different choice situations) or $\mathbf{x}_{ij} \in \mathbb{R}^{d_x}$ (if they vary with choices), parameter vectors $\boldsymbol{\alpha} \in \mathbb{R}^{d_i}$, $\boldsymbol{\beta}, \boldsymbol{\gamma} \in \mathbb{R}^{d_x}$, and random shocks (errors) by ε_{ij}. The total number of products or items is also denoted with $J = |\mathcal{S}|$.

Random Utility Maximization Framework

Under the RUM method [Hess et al., 2018], the utility that individual i gains from alternative $j \in S_i$, denoted as $u_{ij} := u_j(S_i)$, is

$$u_{ij} = v_{ij} + \varepsilon_{ij}, \tag{1.1}$$

where v_{ij} is the deterministic (systematic) component and ε_{ij} is the random component capturing unobserved heterogeneity. Individual i chooses

$$j^* = \underset{j \in S_i}{\mathsf{argmax}}\, u_{ij}. \tag{1.2}$$

The choice probability is

$$\mathbb{P}(i \text{ chooses } j \mid S_i) = \mathbb{P}\left(u_{ij} \geq u_{ik},\ \forall k \in S_i\right). \tag{1.3}$$

Identifying assumptions concern the distribution of ϵ_{ij} and the stability of the distribution of preferences across choice contexts. Classical results characterize when observed choice probabilities are rationalizable by some RUM, through, e.g., block-Marschak inequalities [Block, 1974] and revealed stochastic preference axioms.

Classical Logit-Type Models

Multinomial Logit (MNL)

In the standard MNL model McFadden [1974], ε_{ij}s are i.i.d. Type-I Extreme Value (Gumbel) errors. The systematic utility is typically

$$v_{ij} = \boldsymbol{\beta}^\top \mathbf{x}_{ij}, \tag{1.4}$$

for some parameter vector $\boldsymbol{\beta}$. Then the choice probability is

$$\mathbb{P}(i \text{ chooses } j \mid S_i) = \frac{\exp(v_{ij})}{\sum_{k \in S_i} \exp(v_{ik})}. \tag{1.5}$$

The MNL model is computationally convenient and yields closed-form likelihood, but it implies the independence of irrelevant alternatives (IIA) property: for any $j, k \in S_i$,

$$\frac{\mathbb{P}(j \mid S_i)}{\mathbb{P}(k \mid S_i)} = \frac{\exp(v_{ij})}{\exp(v_{ik})}, \tag{1.6}$$

which does not depend on other alternatives in S_i and can be too restrictive in practice.

Conditional Logit

The conditional logit formulation emphasizes alternative attributes [Train et al., 1987, Train, 2009]. A typical specification is

$$v_{ij} = \boldsymbol{\alpha}^\top \mathbf{x}_i + \boldsymbol{\beta}^\top \mathbf{x}_j + \boldsymbol{\gamma}^\top \mathbf{x}_{ij}, \tag{1.7}$$

where $\mathbf{x}_i$ contains individual-specific features, $\mathbf{x}_j$ fixed item-specific features, and $\mathbf{x}_{ij}$ (item-individual) interactions. The probability, however, retains the classical MNL form. Train et al. [1987] also present an application to local telephone service and calling patterns, illustrating full discrete models of service choice.

Nested Logit

Nested logit relaxes IIA by partitioning alternatives into nests $\{\mathcal{G}_g\}_{g=1}^{G}$ [McFadden, 1978]. For alternative j in nest g, the utility often takes

$$v_{ij} = \boldsymbol{\beta}^\top \mathbf{x}_{ij}, \quad j \in \mathcal{G}_g, \tag{1.8}$$

and the choice probability decomposes as

$$\mathbb{P}(j \mid S_i) = \mathbb{P}(j \mid g, S_i)\, \mathbb{P}(g \mid S_i). \tag{1.9}$$

The within-nest probability is

$$\mathbb{P}(j \mid g, S_i) = \frac{\exp\left(\frac{1}{\lambda_g} v_{ij}\right)}{\sum_{k \in \mathcal{G}_g \cap S_i} \exp\left(\frac{1}{\lambda_g} v_{ik}\right)}, \tag{1.10}$$

and the nest probability is

$$\mathbb{P}(g \mid S_i) = \frac{\exp\left(\lambda_g \, \mathrm{IV}_{ig}\right)}{\sum_h \exp\left(\lambda_h \, \mathrm{IV}_{ih}\right)}, \tag{1.11}$$

with inclusive value

$$\mathrm{IV}_{ig} = \log \sum_{k \in \mathcal{G}_g \cap S_i} \exp\left(\frac{1}{\lambda_g} v_{ik}\right), \tag{1.12}$$

and $\lambda_g \in (0, 1]$ controlling within-nest correlation. McFadden [1978] and Forinash and Koppelman [1993] discuss nested logit for residential location and intercity mode choice.

Latent Class MNL

Latent Class MNL (LC-MNL) models preference heterogeneity via a discrete mixture of MNL components [Greene and Hensher, 2003]. Let $c \in \{1, \ldots, C\}$ index classes, with

class-specific parameters $\boldsymbol{\beta}^{(c)}$. The class-conditional probability is

$$\mathbb{P}(j \mid S_i, c) = \frac{\exp\left((\boldsymbol{\beta}^{(c)})^\top \mathbf{x}_{ij}\right)}{\sum_{k \in S_i} \exp\left((\boldsymbol{\beta}^{(c)})^\top \mathbf{x}_{ik}\right)}. \tag{1.13}$$

Class membership is modeled as

$$\mathbb{P}(c \mid \mathbf{z}_i) = \frac{\exp\left(\boldsymbol{\gamma}_c^\top \mathbf{z}_i\right)}{\sum_{c'=1}^{C} \exp\left(\boldsymbol{\gamma}_{c'}^\top \mathbf{z}_i\right)}, \tag{1.14}$$

where $\mathbf{z}_i$ captures individual covariates. The overall probability marginalizes over classes:

$$\mathbb{P}(j \mid S_i) = \sum_{c=1}^{C} \mathbb{P}(c \mid \mathbf{z}_i)\, \mathbb{P}(j \mid S_i, c). \tag{1.15}$$

Model parameters $\{\boldsymbol{\beta}^{(c)}, \boldsymbol{\gamma}_c\}_{c=1}^{C}$ are typically estimated via EM or direct maximum likelihood.

Interaction-Augmented Models

Halo MNL

Halo MNL [Maragheh et al., 2018] augments MNL to capture pairwise interaction (halo) effects among products. For $j \in S_i$, the deterministic utility is

$$v_{ij} = \boldsymbol{\beta}^\top \mathbf{x}_{ij} + \sum_{k \in S_i,\, k \neq j} h_{jk}, \tag{1.16}$$

where h_{jk} quantifies the (possibly asymmetric) effect of offering product k on the attractiveness of j. The probability retains the MNL form. In matrix notation, let $\mathbf{H} \in \mathbb{R}^{J \times J}$ with $(\mathbf{H})_{jk} = h_{jk}$ and zeros on the diagonal (J is the total number of items). Then, for a given choice set, interactions correspond to row sums of $\mathbf{H}$ restricted to S_i.

Parameter identifiability requires structural constraints on $\mathbf{H}$, and the total number of interaction parameters is $O(J^2)$ for J products, implying $\Omega(J^2)$ samples for reliable estimation. Halo MNL captures complementarities and substitution patterns but can overfit in large assortments.

Low-Rank Halo / Self-Attention Choice Models

Ko and Li [2024] show that Halo MNL can be generalized via a low-rank factorization

$$h_{jk} = \mathbf{u}_j^\top \mathbf{v}_k, \quad \mathbf{u}_j, \mathbf{v}_k \in \mathbb{R}^r,\ r \ll J, \tag{1.17}$$

dramatically reducing the number of free parameters from $O(J^2)$ to $O(Jr)$. They implement this with self-attention:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right), \tag{1.18}$$

and the context-augmented representations are

$$\tilde{\mathbf{X}} = \mathbf{A}\mathbf{V}. \tag{1.19}$$

where rows of $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are learned embeddings of alternatives. The resulting utility for j becomes

$$v_{ij} = \boldsymbol{\beta}^\top \mathbf{x}_{ij} + f_{\text{att}}(\tilde{\mathbf{x}}_{ij}, \boldsymbol{\theta}), \tag{1.20}$$

with f_{att} parameterized by attention weights. Ko and Li [2024] provide theoretical guarantees: under mild conditions, their self-attention estimator achieves near-optimal stationary points with $O(J)$ samples, a polynomial improvement over Halo MNL. This yields a scalable interaction-aware logit model for large assortments.

Neural Discrete Choice Models

RUMnet

RUMnet [Aouad and Désir, 2023] represents RUM-consistent choice models with neural networks. For each alternative j, consider T samples of latent utility:

$$u_{ij}^{(t)} = f_j(\mathbf{x}_{ij}; \boldsymbol{\theta}^{(t)}) + \varepsilon_{ij}^{(t)}, \quad t = 1, \ldots, T, \tag{1.21}$$

with f_j parameterized by neural networks and $\boldsymbol{\theta}^{(t)}$ representing sampled network parameters or input noise. The choice probability is approximated via sample average:

$$\mathbb{P}(j \mid S_i) \approx \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}\Big(u_{ij}^{(t)} \geq u_{ik}^{(t)},\ \forall k \in S_i\Big). \tag{1.22}$$

Aouad and Désir [2023] prove that RUMnets can approximate any RUM-derived choice model arbitrarily well with sufficient capacity and samples, and that any RUMnet is rationalizable by some RUM. They also derive generalization bounds depending on $\|\boldsymbol{\theta}\|$, sample size, and architecture depth/width.

TasteNet-MNL

TasteNet [Han et al., 2022] embeds a neural network into MNL to learn taste heterogeneity. Let $\mathbf{z}_i$ denote socio-demographic features. TasteNet parametrizes individual-level

coefficients as

$$\boldsymbol{\beta}_i = g(\mathbf{z}_i; \boldsymbol{\theta}_{\text{taste}}), \tag{1.23}$$

where g is a feed-forward network. The utility is

$$v_{ij} = \boldsymbol{\beta}_i^\top \mathbf{x}_{ij}, \tag{1.24}$$

and the choice probability is standard MNL. The key idea is to model $\boldsymbol{\beta}_i$ flexibly as a nonlinear function of $\mathbf{z}_i$, capturing rich heterogeneity while retaining interpretable $\boldsymbol{\beta}_i$ as marginal sensitivities.

On synthetic data, TasteNet-MNL recovers ground-truth nonlinear mappings between $\mathbf{z}_i$ and $\boldsymbol{\beta}_i$. On the Swissmetro dataset [Bierlaire, 2001], it outperforms classic MNL while revealing more realistic value-of-time distributions.

Learning-MNL

Learning-MNL [Sifringer et al., 2020] decomposes systematic utility into a knowledge-driven and a learned component:

$$v_{ij} = v_{ij}^{\text{KN}} + v_{ij}^{\text{NN}}, \tag{1.25}$$

with

$$v_{ij}^{\text{KN}} = \boldsymbol{\beta}^\top \mathbf{x}_{ij}, \quad v_{ij}^{\text{NN}} = h(\mathbf{x}_{ij}; \boldsymbol{\theta}_{\text{rep}}), \tag{1.26}$$

and h implemented by a neural network. The choice probability is MNL with v_{ij} above. This structure leverages domain knowledge via V^{KN} while allowing V^{NN} to correct misspecification. Estimation is via joint maximum likelihood over $(\boldsymbol{\beta}, \boldsymbol{\theta}_{\text{rep}})$, with regularization on $\boldsymbol{\theta}_{\text{rep}}$ to mitigate overfitting and preserve interpretability of $\boldsymbol{\beta}$.

ResLogit

ResLogit [Wong and Farooq, 2021] uses residual neural networks to model v_{ij}:

$$\mathbf{v}_{ij}^{(0)} = \mathbf{x}_{ij}, \quad \mathbf{v}_{ij}^{(\ell+1)} = \mathbf{v}_{ij}^{(\ell)} + F^{(\ell)}(\mathbf{v}_{ij}^{(\ell)}; \boldsymbol{\theta}^{(\ell)}), \tag{1.27}$$

for $\ell = 0, \ldots, L-1$, and

$$v_{ij} = \mathbf{w}^\top \mathbf{v}_{ij}^{(L)}. \tag{1.28}$$

Residual mappings $F^{(\ell)}$ are typically shallow networks. The ResNet structure facilitates training deep architectures by preserving gradient flow. As in MNL,

$$\mathbb{P}(j \mid S_i) = \frac{\exp(v_{ij})}{\sum_{k \in S_i} \exp(v_{ik})}. \tag{1.29}$$

Wong and Farooq [2021] empirically show that ResLogit outperforms shallower NN-logit and classical models on transport datasets, especially when utility is highly nonlinear in $\mathbf{x}_{ij}$.

Context-Dependent Models

Deep Context-Dependent and RKHS Choice Models

Deep context-dependent choice models [Zhang et al., 2025] decompose the systematic utility of j as

$$v_{ij} = v_j(S_i) = v_j + \sum_{T \subseteq S_i \setminus j} v_j(T), \tag{1.30}$$

where v_j and $v_j(T)$ are parameterized via permutation-equivariant neural networks. This allows explicit control over the order of interaction (pairwise, triplets, etc.) and preserves exchangeability over alternatives.

RKHS-based models [Yang et al., 2025] embed $\mathbf{X}_{S_i} = (\mathbf{x}_i, \mathbf{x}_{ij})$ into a reproducing kernel Hilbert space $(\mathcal{H}, \langle \cdot, \cdot \rangle_{\mathcal{H}})$ with kernel $\bar{\mathbf{K}}(\cdot, \cdot)$, and represent systematic utility as

$$v_{ij} = \mathbf{e}_j^\top \sum_{n=1}^{N} \bar{\mathbf{K}}\big((\mathbf{X}_{S_i}, S_i), (\mathbf{X}_{S_{i_n}}, S_{i_n})\big)\boldsymbol{\alpha}_n, \tag{1.31}$$

with coefficients $\{\boldsymbol{\alpha}_n\}$. Regularization in $\mathcal{H}$ (e.g. via $\|\mathbf{v}\|_{\mathcal{H}}^2$) controls complexity and mitigates overfitting. These methods will be further discussed in Chapter 2.

Sparse Market-Product Shocks and Endogeneity

When analyzing market-level aggregated data, unobserved product characteristics $\boldsymbol{\xi}_{jt}$ can be correlated with prices, creating endogeneity bias. Berry et al. [1995] pioneered the BLP approach using instrumental variables.

Recent work by Lu and Shimizu [2025] extends the BLP framework by exploiting sparsity in market-product shocks, providing alternatives to instrumental variable assumptions when exogenous instruments are weak or unavailable. They use a random-coefficient logit model for market-product-level choices, where the utility of consumer i for product j in market t is modeled as

$$u_{ijt} = \mathbf{x}_{jt}^\top \boldsymbol{\beta}_i + \xi_{jt} + \varepsilon_{ijt}.$$

Here, $\mathbf{x}_{jt}$ are observed characteristics, $\boldsymbol{\beta}_i$ are heterogeneous tastes, and ξ_{jt} is the unobserved market–product-level demand shock which is assumed to vary sparsely between different products in a market ($\xi_{jt} = \bar{\xi}_t + \eta_{jt}$ where η_{jt} is sparse). We will discuss this model in more details in Chapter 3.

Based on this literature review, we will now address the questions posed in the credit card offer demand estimation challenge. The corresponding answers are provided in two chapters corresponding to parts one and two of the challenge, given below.

Chapter 2

Part 1: Discrete Context-Dependent Choice Model

a Errors or Unclear Parts in [Zhang et al., 2025]

The following is a list of typos, mathematical and structural errors, and unclear parts in the paper:

- In section 2.1, when the authors define $P_j(S)$, they say that $u_j(T)$ represents the utility of item j when the context subset is $T \subseteq S \backslash \{j\}$ (and hence, T should not contain j. But later in the definition of $P_j(S)$, they say that $j \in S$ and they use the notation $u_j(S)$ which is a little bit confusing. The correct form is to say $u_j(T)$ represents the utility of item j in set $T = \{j, T'\} \mid T' \subseteq S \backslash \{j\}$.

- In section 3, in the first paragraph it is told that $\mathbf{x}_j \in \mathbb{R}^{d_x}$, i.e., each item (alternative) has d_x-dimensional feature. However, later d_x is mixed with d, like for example in the definition of $\mathbf{X}_S$ which should be of size $d_x \times J$ not $d \times J$.

- In equation (3), the notation used for $u_j(\mathbf{x}_j, \mathbf{X}_R)$ should be $u_j(\mathbf{X}_{\{j\} \cup R})$ to be consistent with the notation in equation (2). Moreover, the representation seems a little bit different from that given in equation (2). In that equation, $v_j(\phi)$ represents the intrinsic utility of alternative j, while in equation (3) $v_j(\mathbf{x}_j)$ represents the same value. Additionally, the notation $\subset$ should be replaced with $\subseteq$.

- The authors' proposal to capture higher-order context effects via the neural network design in equations (4) and (5) is less explicit than the feature concatenation approach in Pfannschmidt et al. [2022]. However, their approach is more computationally feasible; as with increasing network depth (interaction order), the size of the network will exponentially increase with feature concatenation.

- On page 4, column 2, nonlinear embedding function should be $\chi : \mathbb{R}^{d_x} \to \mathbb{R}^d$ (replacing d_0 with d) to be consistent with the remaining notations.

- The function Φ_h^l in equation (7) should be a polynomial function on $\mathbb{R} \times \mathbb{R}^d$ (not $\mathbb{R} \times \mathbb{R}$).

- Equation (8) is incorrect. It is unclear how setting $\sigma(\cdot)$ as a quadratic function and $\Phi_h^l(\cdot, \cdot)$ as a linear transformation of its second argument leads to this equation. Apparently, $\mathbf{z}_j^0$ should appear in equation (8). Let $\mathbf{W}^l \in \mathbb{R}^{H \times d}$. Also assume $\Phi_h^l(x, \mathbf{y}) = f(x)\mathbf{y}$. Then, according to equations (4) and (5) in Zhang et al. [2025], we have:

$$\bar{\mathbf{z}}^l = \mathbf{W}^l \left(\frac{1}{S} \sum_{k=1}^{S} \sigma(\mathbf{z}_k^{l-1}) \right), \tag{2.1}$$

$$\mathbf{z}_j^l = \mathbf{z}_j^{l-1} + \frac{1}{H} \sum_{h=1}^{H} f(\bar{z}_h^l) \phi^l(\mathbf{z}_j^0). \tag{2.2}$$

  The last equation is clearly different form what is given in (8). This equation may only hold under a specific condition. For example, if $\mathbf{z}_j^0 = \mathbf{e}_j$, $\phi^l(\mathbf{z}_j^0) = \mathbf{e}_j$, $H = J$, and $f(\bar{z}_h^l) = \bar{z}_h^l$, then we get:

$$\mathbf{z}_j^l = \mathbf{z}_j^{l-1} + \frac{1}{J} \sum_{j=1}^{J} \bar{z}_j^l \mathbf{e}_j = \mathbf{z}_j^{l-1} + \frac{1}{J} \bar{\mathbf{z}}^l = \mathbf{z}_j^{l-1} + \frac{1}{S} \sum_{k=1}^{S} (\mathbf{W}^l / J) \sigma(\mathbf{z}_k^{l-1}). \tag{2.3}$$

  Note: Comparing the last equation on page 4 of Zhang et al. [2025] with equation (5), one may think there is an error in equation (5) and $\phi_h^l(\mathbf{z}_j^0)$ should be replaced with $\phi_h^l(\mathbf{z}_j^{l-1})$. However, equation (5) is already correct. This equation states that the hidden state (variable) from previous layer $\mathbf{z}_j^{l-1}$ is combined with the initial influence of each alternative $\phi_h^l(\mathbf{z}_j^0)$, through modulation with head-specific coefficients of the context summary from the previous layer $(\bar{z}_h^l)$, to produce the hidden state for the current layer $\mathbf{z}_j^l$. Hence, with $\mathbf{z}_j^1$ capturing the first order halo effects, i.e., effect of $\{k\}$, $k \in S \setminus \{j\}$ on j, then $\mathbf{z}_j^2$ captures the second order halo effects (effect of $\{k, l\}$, $k, l \in S \setminus \{j\}$ on j).

- The authors propose the following decomposition for the utility function:

$$u_j(\mathbf{X}_S) = \sum_{p=0}^{|S|-1} u_j^{(p)}(\mathbf{X}_S), \tag{2.4}$$

  where $u_j^{(p)}(\mathbf{X}_S)$ denotes the contribution of the p-th order interactions and is defined

as

$$u_j^{(p)}(\mathbf{X}_S) := \sum_{T \subseteq S \setminus \{j\}, |T|=p} v_j(\mathbf{X}_{T \cup \{j\}}). \tag{2.5}$$

However, the authors do not clarify how the marginal utility terms $v_j(\mathbf{X}_{T\cup\{j\}})$ are obtained. Furthermore, in the last paragraph of page 4 of Zhang et al. [2025], it is suggested that $u_j^{(p)}(\mathbf{X}_S) = \boldsymbol{\beta}^\top \mathbf{z}_j^p$. Given this and using (2.4), we yield:

$$u_j(\mathbf{X}_S) = \sum_{p=0}^{|S|-1} \boldsymbol{\beta}^\top \mathbf{z}_j^p. \tag{2.6}$$

However, later, on page 5 (after equation (5)), the final utility function is given as $u_j(\mathbf{X}_S) = \boldsymbol{\beta}^\top \mathbf{z}_j^L$, which is in contradiction with (2.6) (if $L = |S| - 1$). Hence, the p-th order interaction cannot be simply formulated as $u_j^{(p)}(\mathbf{X}_S) = \boldsymbol{\beta}^\top \mathbf{z}_j^p$.

Note: In our implementation, we obtain the final utility as $u_j(\mathbf{X}_S) = \boldsymbol{\beta}^\top \mathbf{z}_j^L$, where L is the last layer index.

- A utility function $u_j(\mathbf{X}_S)$ is permutation equivariant if $u_j(\mathbf{X}_{\pi(S)}) = u_{\pi(j)}(\mathbf{X}_S)$. The featured implementation of the DeepHalo method, based on equations (4) and (5) of Zhang et al. [2025], satisfies permutation equivariance. The context vector $\bar{\mathbf{z}}^l$ in equation (2.1) is clearly permutation equivariant as it is a linear function of a (symmetric) sum over $\mathbf{z}_k^{l-1}$. For each item, the update $\mathbf{z}_j^l$ also only depends on $\mathbf{z}_j^{l-1}$ and $\mathbf{z}_j^0$ which represent the embeddings of the same item. Hence, the featured implementation is permutation equivariant. However, the featureless implementation based on equation (10) of Zhang et al. [2025] is not permutation equivariant. To prove that, let $\mathbf{P} \in \{0,1\}^{J\times J}$ be a permutation matrix corresponding to the permutation π. We need to show that $\mathbf{u}(\mathbf{X}_S\mathbf{P}) = \mathbf{P}\mathbf{u}(\mathbf{X}_S)$. Consider equation (10) in Zhang et al. [2025] as

$$\mathbf{y}^l = \mathbf{y}^{l-1} + \boldsymbol{\Theta}^l \sigma(\mathbf{y}^{l-1}), \tag{2.7}$$

which starts from $\mathbf{y}^0(\mathbf{X}_S) = \sum_j \mathbf{x}_j = \mathbf{X}_S\mathbf{1}$. Let $\mathbf{u}(\mathbf{X}_S) = \mathbf{y}^L(\mathbf{X}_S)$. Then, we have to verify the equality $\mathbf{y}^L(\mathbf{X}_S\mathbf{P}) = \mathbf{P}\mathbf{y}^L(\mathbf{X}_S)$. It is straightforward to show that $\mathbf{y}^0(\mathbf{X}_S)$ is invariant to the permutation of the columns (items) of $\mathbf{X}_S$ as

$$\mathbf{y}^0(\mathbf{X}_S\mathbf{P}) = \mathbf{X}_S\mathbf{P}\mathbf{1} = \mathbf{X}_S\mathbf{1} = \mathbf{y}^0(\mathbf{X}_S). \tag{2.8}$$

Hence, $\mathbf{y}^L(\mathbf{X}_S) = \mathbf{y}^L(\mathbf{X}_S\mathbf{P})$ (since the initial point is the same in both cases). This implies invariance of the utility function to the permutation of the items, which is different from equivariance. The latter requires $\mathbf{y}^L(\mathbf{X}_S\mathbf{P}) = \mathbf{P}\mathbf{y}^L(\mathbf{X}_S)$ which does not hold.

- Calculation of the relative context effect $\alpha_{jk}(T)$ requires:

$$\alpha_{jk}(T) = [v_j(T) + v_j(T \cup \{k\})] - [v_k(T) + v_k(T \cup \{j\})]. \tag{2.9}$$

  Each $v_j(T)$ needs $2^{|T|}$ evaluations via (3) as

$$v_j(\mathbf{X}_{T\cup\{j\}}) = \sum_{R\subseteq T} (-1)^{|T|-|R|} u_j(\mathbf{X}_{\{j\}\cup R}), \tag{2.10}$$

  which is infeasible for large choice sets, when $|S| \gg 1$.

b Replication of Synthetic Data Test in [Zhang et al., 2025]

b.1 A Short Tutorial on Choice-Learn Python Package

The `choice-learn` Python package is a modular framework for estimating discrete choice models supporting both single and multiple choice (basket) models . This package is based on Tensorflow and is designed for optimized Data handling with the `ChoiceDataset` class and ready-to-use (as well as customized) models with the `ChoiceModel` class.

The data object instantiated by the `ChoiceDataset` class is characterized by four main tensors [Auriau et al., 2024]:

- `shared_features_by_choice`: context- or decision-maker-specific features (a matrix of shape $(n_{\text{choices}}, n_{\text{shared-features}})$).
- `items_features_by_choice`: alternative- and interaction-specific features (a matrix of shape $(n_{\text{choices}}, n_{\text{items}}, n_{\text{items-features}})$).
- `available_items_by_choice`: availability indicators for each alternative in each choice set (of shape $(n_{\text{choices}}, n_{\text{items}})$).
- `choices`: the index of the chosen alternative for each choice (of shape $(n_{\text{choices}}, n_{\text{items}})$).

On top of this data layer, the package implements a set of models built upon the base class `ChoiceModel`, which provides a common interface for specification, prediction, and learning of a choice model [Auriau et al., 2024].

The `ChoiceModel` base class provides several methods and attributes that every model (built-in or custom) is expected to use. Model specification occurs in `__init__()` and optionally `instantiate()`, where hyperparameters such as optimizer, learning rate, and number of epochs are set. Model estimation is handled by the `fit()` method, which takes a `ChoiceDataset`, computes the negative log-likelihood of the implied choice probabilities, and runs the chosen TensorFlow optimizer (by default `lbfgs` for small to medium

problems, or stochastic gradient optimizers such as Adam for large-scale data) [Auriau et al., 2024]. Once trained, the model can be used via `evaluate()` (to compute average negative log-likelihood on a dataset), `predict_probas()` (to obtain choice probabilities $\hat{P}(j \mid S_i)$ for each choice situation), and `compute_batch_utility()` (to obtain the matrix of utilities for a batch of choices and items.

Custom models are created by subclassing `ChoiceModel` and implementing two main elements: (i) the initialization of trainable parameters in `__init__()`, and (ii) the utility computation in `compute_batch_utility()`, which receives four arguments: `shared_features_by_choice`, `items_features_by_choice`, `available_items_by_choice`, and `choices`. The output must be a tensor of shape $(n_{\text{choices}}, n_{\text{items}})$ containing the systematic utility u_{ij} of each item for each choice in the batch.

For example, in Train's formulation [Train, 2009] for the utility of alternative j in choice set i as

$$u_{ij} = \boldsymbol{\alpha}^\top \mathbf{x}_i + \boldsymbol{\beta}^\top \mathbf{x}_j + \boldsymbol{\gamma}^\top \mathbf{x}_{ij},$$

$\mathbf{x}_i$ denotes `shared_features_by_choice`, while the vectors $\mathbf{x}_j$ and $\mathbf{x}_{ij}$ are both represented with `items_features_by_choice`, where item-only attributes (corresponding to $\mathbf{x}_j$) are repeated across choices when they are time-invariant, and interaction attributes (corresponding to $\mathbf{x}_{ij}$) are defined at the (i, j) level as additional feature columns for each item in each choice. The fields `choices` and `available_items_by_choice` then specify, for each choice situation i, which alternatives $j \in S_i$ were available and which alternative was actually chosen.

b.2 Our Contribution

In the main formulation of the paper, the input feature matrix $\mathbf{X}_S$ is constructed as

$$\mathbf{X}_S = [\mathbf{x}_1, \ldots, \mathbf{x}_{|S|}, \mathbf{O}_{J-|S|}] \in \mathbb{R}^{d_x \times J} \tag{2.11}$$

where $\mathbf{O}_{J-|S|} \in \mathbb{R}^{d_x \times (J-|S|)}$ is a matrix of all zeros. In other words, the right-most columns of the matrix corresponding to the indices of the unavailable items are always padded with zeros. This requires rearranging the columns of the feature matrix (resorting $\mathcal{S} = \{1 \ldots, J\}$), so that the first $|S|$ columns represent available items features. We revised the code so that it works with fixed data features (fixed item indices) and availability mask, instead of resorting the feature matrix to have available items at the beginning and giving the lengths of the available items. In our implementation, the columns of the matrix are fixed and we instead, mask the feature matrix with the availability input

vector $\boldsymbol{\mu} \in \{0,1\}^J$. Then, equation (4) becomes:

$$\bar{\mathbf{z}}^l = \frac{1}{|\sum_j \mu_j|} \mathbf{W}^l \sum_{k=1}^{J} \mu_k \mathbf{z}_k^{l-1} \tag{2.12}$$

Our choice is more convenient and generalizable.

We have developed a python package, named `DeepHalo`, for the implementation of the deep context-dependent choice model of Zhang et al. [2025] in `choice-learn`. We implemented two network setups addressing the featured and featureless data scenarios (especially for the synthetic experiment in section 5.1 of the paper). The featured model can be built from the `DeepHaloFeatured` class in `Featured_DeepHalo` python module. The featureless model can also be instantiated from `DeepHaloFeatureless3D` class inside the `Featureless_DeepHalo` module. The neural network implementation in feature-based setting is based on equations (15) and (16) in Section B of [Zhang et al., 2025]. The original featureless model in, based on formulations given in Section 4.2 of [Zhang et al., 2025], is not permutation equivariant (this is discussed in the previous answer). Hence, we implemented a corrected version in the `DeepHaloFeatureless3D` model which deals with 3D one-hot encoded $\mathbf{1}_j(S) = \mathbb{1}(j \in S)$ item features. Here, instead of initial global sum pooling of features tensor as proposed in Zhang et al. [2025], we apply per-item transformations that preserve equivariance. The implementation is indeed, similar to the featured model. The difference is that here we choose initial identity map for $\mathcal{X}$ and linear head-specific transformations for $\phi_h^l(\mathbf{z}^0)$.

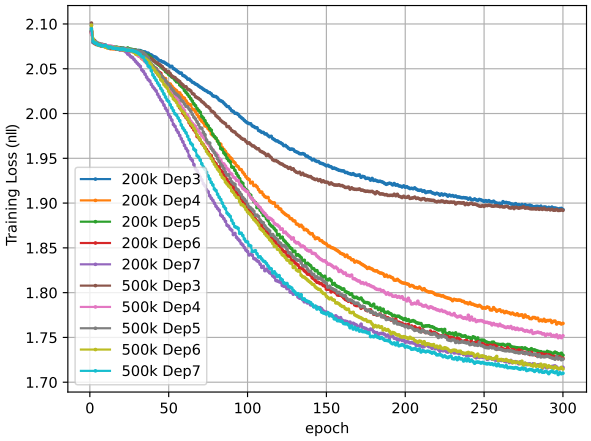
To implement our models (both featured and featureless versions) with `choice-learn`, we inherit from the `SimpleMNL` class in the `choice_learn.models` module. We use `Adam` as the default optimizer and we choose negative log-likelihood for the loss function (MSE can also be chosen by changing the `loss_name` attribute of the developed `DeepHaloChoiceModel` model. We pass:

- `items_features_by_choice`: 3D tensor of shape (B, J, D), where B is the batch-size,
- `available_items_by_choice`: 2D matrix of shape (B, J) for the availability mask,
- `choices`: 1D vector of shape $(B,)$ for the choices,

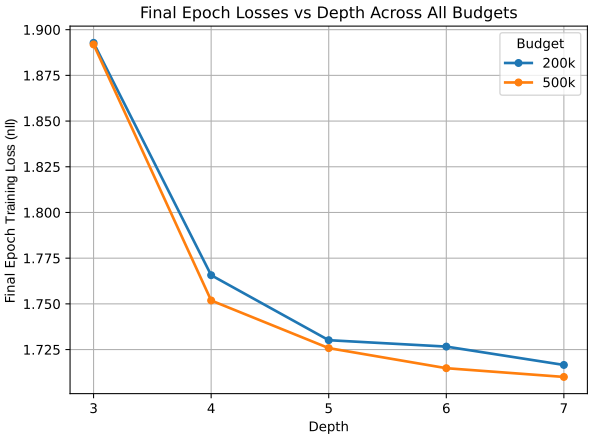
to the `compute_batch_utility()` method. This function is subsequently used for training and prediction in the `SimpleMNL` model.

To replicate the synthetic data results in Zhang et al. [2025], we consider the featureless model with the following experimental setup:

- Different network depth: $L \in \{3, 4, 5, 6, 7\}$,
- Different parameter budgets: budget $\in \{200000, 500000\}$,



(a) Training loss (negative log-likelihood) curve across epochs for different parameter budgets



(b) Final training loss vs. network depth

Figure 2.1: Evaluating the effect of network depth (Halo effect order) on training loss using a synthetic choice dataset with $S = 15$ and $J = 18$.

- Different attention heads H: for each parameter budget, we find a depth-width pair (d, H) such that the number of trainable parameters of the network matches the budget (we also set the embedding dimension to $d = J$).
- Quadratic activation: similar to the synthetic data test in [Zhang et al., 2025], we apply a quadratic aggregation to the embeddings.

For data generation, we perform the following[1]:

1. Choose $N = \binom{J}{S}$ items for the choice set (each with exactly S alternatives). We choose $S = 15$ and $J = 18$.

   Note: For $J = 20$ (as with the original data test in [Zhang et al., 2025]), the dataset would become excessively large, causing memory exhaustion on our system

[1] https://github.com/Asimov-Chuang/DeepHalo

during training with `choice-learn`. Hence, we instead tested on $J = 18$ for reduced complexity.)

2. Sample choice probability vectors uniformly from the simplex on $\mathbb{R}^S$
3. For each choice set with S items, generate 40 i.i.d. choice observations for training and 10 i.i.d. observations for test

Fig. 2.1 shows the numerical result for the featureless model of Zhang et al. [2025] over a synthetic choice dataset with $S = 8$ alternatives in every choice set, from a universe of $J = |\mathcal{S}| = 15$ items. Data generation process is similar to the experimental setup given in Section 5.1 of [Zhang et al., 2025] for "Synthetic Data with High-order Effects". To evaluate the effect of depth on Halo order modeling, we change the depth (number of layers) of the network, while fixing the total number of parameters (budget). This is to avoid misinterpretation due to varying model complexity. We plot the negative log-likelihood loss curves over training data for different choices of the network depth $L \in \{3, \ldots, 7\}$ (with H being adjusted accordingly). As shown in this figure, the loss decreases as depth increases up to 5, beyond which 2^{5-1} exceeds the maximum interaction order $S = 15$. Hence, the results align closely with those reported in [Zhang et al., 2025].

c Additional Tests for Verification

The key result in [Zhang et al., 2025] is that with quadratic activations, an L-layer network can represent interactions up to order 2^{L-1}.

As mentioned in the previous answer, this was actually validated through the replicated experimental setup on synthetic data with $J = 18$ and $S = 15$.

To further verify this bound, we perform additional tests on a variety of experimental setups involving both synthetic and real data described below.

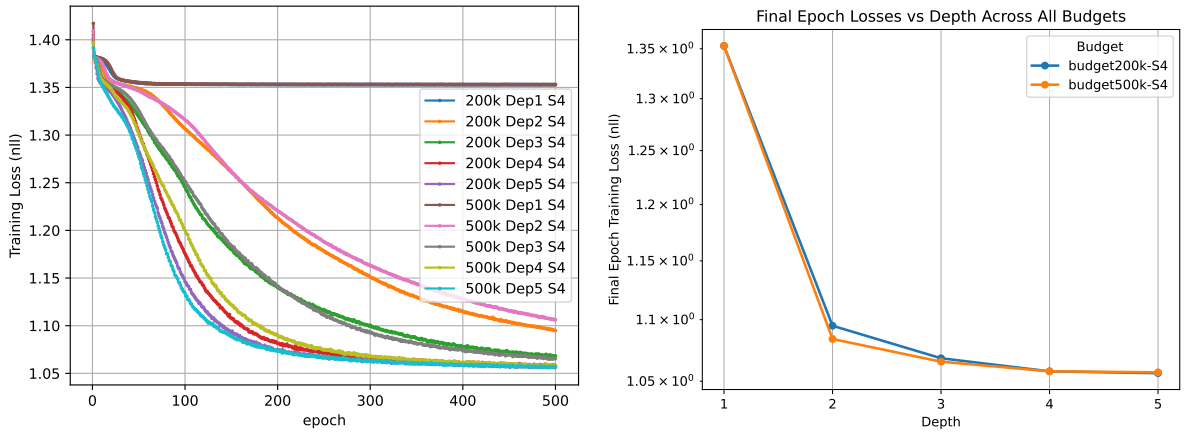
c.1 Synthetic Data Tests: Featureless Setting

Here, we conduct additional tests on synthetically generated data with different choice set sizes S and different universe of alternatives with cardinality J, to verify the theoretical 2^{L-1} order bound. The synthetic data is generated similarly to the previous section. We consider:

- Different network depth: $L \in \{3, 4, 5, 6, 7\}$ if $S > 4$ and $L \in \{1, 2, 3, 4, 5\}$ if $S \leq 4$
- Different choice set/ universe of alternatives cardinality:
  $(S, J) \in \{(4, 12), (8, 12), (16, 19), (32, 34)\}$

For each (S, J) pair, we perform the following:

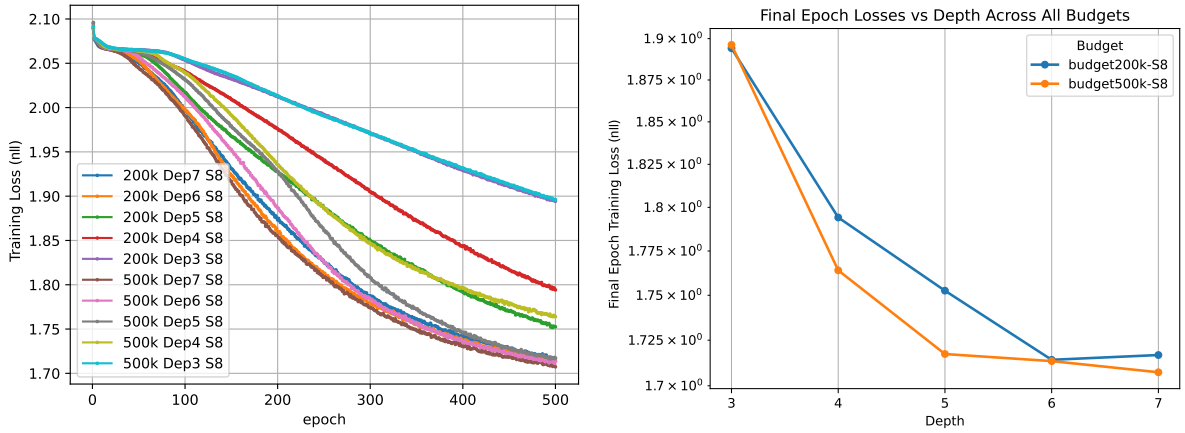
1. Choose $N = \binom{J}{S}$ (available) items for the choice set (each with exactly S alternatives)
2. Sample choice probability vectors uniformly from the simplex on $\mathbb{R}^S$
3. For each choice set, generate 40 i.i.d. choice observations for training and 10 i.i.d. observations for test
4. For a fixed parameter budget, we adjust the the number of interaction heads H such that the number of trainable parameters of the network matches the budget (we also set the embedding dimension to $d = J$).



(a) Training loss evolution across epochs for models with varying parameter budgets.

(b) Final training loss vs. network depth

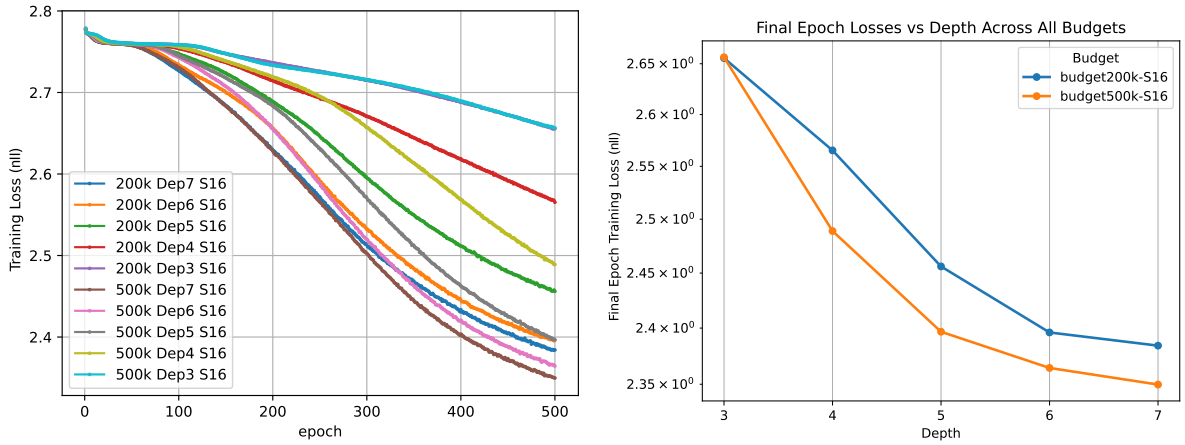
Figure 2.2: Evaluating the effect of network depth (Halo effect order) on training loss using a synthetic choice dataset with $(S = 4, J = 12)$. The results are based on the equivariant featureless model `DeepHaloFeatureless3D`.



(a) Training loss evolution across epochs for models with varying parameter budgets.

(b) Final training loss vs. network depth

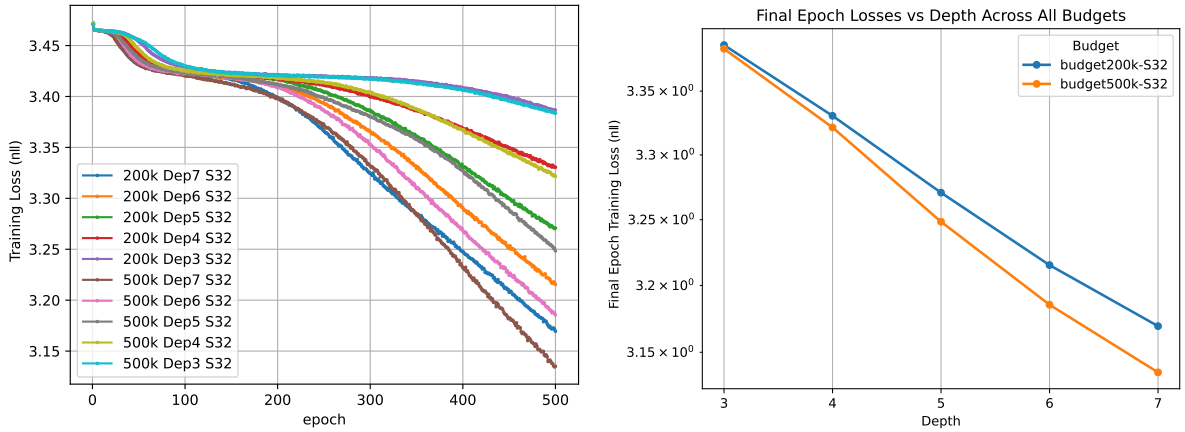
Figure 2.3: Evaluating the effect of network depth (Halo effect order) on training loss using a synthetic choice dataset with $(S = 8, J = 12)$. The results are based on the equivariant featureless model `DeepHaloFeatureless3D`.



(a) Training loss evolution across epochs for models with varying parameter budgets.

(b) Final training loss vs. network depth

Figure 2.4: Evaluating the effect of network depth (Halo effect order) on training loss using a synthetic choice dataset with $(S = 16, J = 19)$. The results are based on the equivariant featureless model `DeepHaloFeatureless3D`.

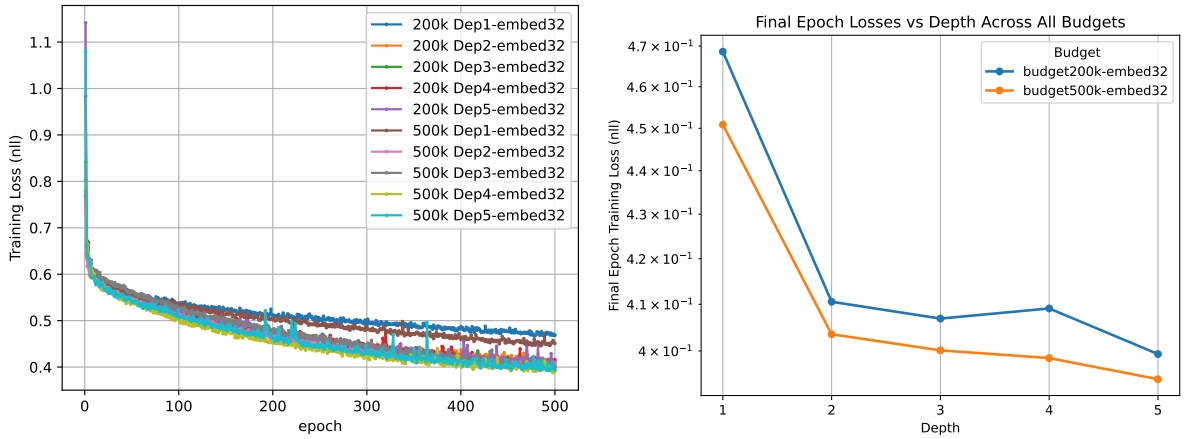


(a) Training loss evolution across epochs for models with varying parameter budgets.

(b) Final training loss vs. network depth

Figure 2.5: Evaluating the effect of network depth (Halo effect order) on training loss using a synthetic choice dataset with $(S = 32, J = 34)$. The results are based on the equivariant featureless model `DeepHaloFeatureless3D`.

Our hypothesis is that training loss should decrease as depth increases until $S - 1 \leq 2^{L-1}$ (i.e., network depth matches or exceeds the maximum interaction order in the choice sets). Beyond this threshold, no significant improvement should occur; the plot of final loss versus depth should begin to flatten. This behavior is visible in Figures 2.3, 2.4, and 2.5, which plot the training loss (negative log-likelihood) versus depth for synthetic datasets with $(S = 4, J = 12)$, $(S = 8, J = 12)$, $(S = 16, J = 19)$, and $(S = 32, J = 34)$, respectively. Here we implemented the `DeepHaloFeatureless3D` model with quadratic activation (for $\sigma(\cdot)$ in equation (6) of [Zhang et al., 2025]) and embedding dimension $d = J$. While loss improvement begins to diminish beyond a certain depth, that depth



(a) Training loss evolution across epochs for models with varying parameter budgets.

(b) Final training loss vs. network depth

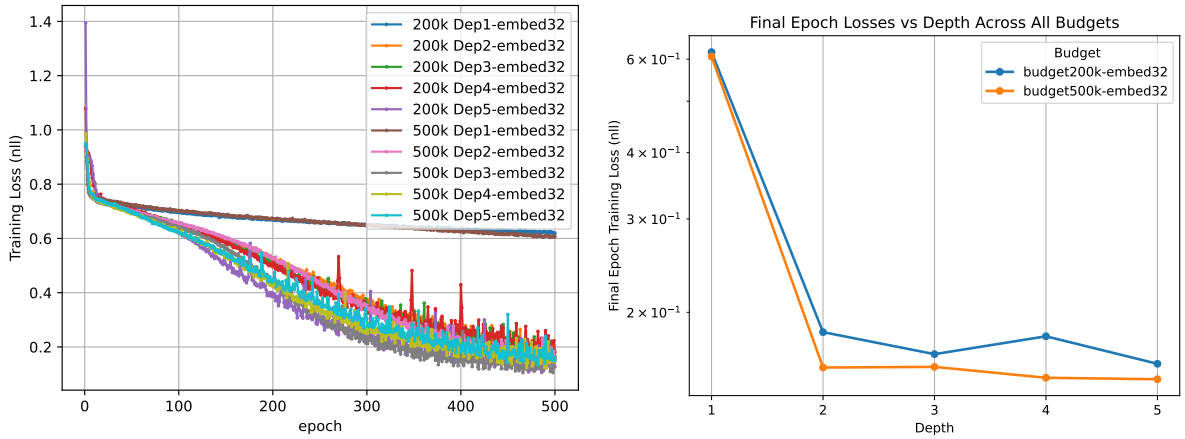
Figure 2.6: Training loss vs. network depth, evaluated on the (real) ModeCanada transportation choice dataset with $J = 4$ items. The results are based on the `DeepHaloFeatured` model with an embedding dimension of $d = 32$.

appears to be one layer more than the theoretical threshold $\lceil \log_2(S-1) \rceil + 1$. This is perhaps because the chosen choice-set sizes are exact powers of two. Nevertheless, these experiments confirm that the network's expressiveness clearly depends on its depth.

c.2 Real Data Tests: Featured Setting

Next, we test our models using real-world benchmark data with item-level features. For this, we use the transportation mode-choice datasets included in the `choice-learn` package: **ModeCanada** and **SwissMetro**.

ModeCanada (or the Montreal-Toronto Corridor Mode Choice dataset) records individuals' choices among travel modes in Canada including air, train, bus, and car ($J = 4$ items) with item features such as cost, frequency, in-vehicle and out-of-vehicle time, and shared features like trip distance, income, and urban status. SwissMetro is a stated-preference dataset about hypothetical Swiss transport modes (TRAIN, SwissMetro, CAR) with $J = 3$ items, with item features such as availability, travel time, and cost, and shared features like purpose and age. For our experiment, we exclude the shared features and only provide our models with item features, availability mask and the choices. We choose `'TT'` (travel time) and `'CO'` (cost) as features for the SwissMetro dataset and `'freq'` (frequency of service), `'cost'`, `'ovt'` (out of vehicle time), and `'ivt'` (in-vehicle time) as the items features for ModeCanada dataset. Fig. 2.6 and Fig. 2.8 show the results of our `DeepHaloFeatured` model on the ModeCanada and SwissMetro datasets, respectively. As these figures show, for depths beyond $\lceil \log_2(S-1) \rceil + 1$ (which equals 3 for ModeCanada dataset with $S \leq 4$ and 2 for SwissMetro dataset with $S \leq 3$), the training loss improves very little. The fluctuations in these figures arise because the



(a) Training loss evolution across epochs for models with varying parameter budgets.

(b) Final training loss vs. depth

Figure 2.8: Training loss vs. network depth, evaluated on the (real) SwissMetro transportation choice dataset with $J = 3$ items. The results are based on the `DeepHaloFeatured` model with an embedding dimension of $d = 32$.

number of parameters is not exactly fixed; with an embedding size of $d = 32$, the exact expected budget cannot be met precisely and may slightly vary with depth.

c.3 Other Tests

We have additionally provided a test suite with `unittest` (within the `tests` directory of the provided `DeepHalo` package) to evaluate different aspects of the designed models including initialization, inheritance, performance, and properties. We particularly test equivariance of the designed models to the permutation of the alternatives in a choice set. As previously discussed, with 3D items features, the featured and featureless implementations of the DeepHalo method based on equations (4) and (5) of Zhang et al. [2025], respectively represented by `DeepHaloFeatureless3D` and `DeepHaloFeatured` models, are permutation equivariant. However, this property does not hold for the original featureless version, which is based on the formulation in equation (10) of Zhang et al. [2025]. This featureless model is, instead, permutation invariant. To verify this, we have also implemented the original featureless version inside the `DeepHaloChoiceModel`. It can be called using the `DeepHaloFeatureless2D` class. The tests can now verify that the original model is permutation invariant. These tests can be simply executed with the `DeepHalo_Tests.py` python module inside the `tests` directory.

d Comparison between [Yang et al., 2025] and [Zhang et al., 2025]

d.1 A Summary of [Yang et al., 2025]

Yang et al. [2025] introduce a novel vector-valued Reproducing Kernel Hilbert Space (RKHS) framework for discrete choice modeling.

It assumes that the utility function $\mathbf{u}$ lies in a $\mathbb{R}^J$ vector-valued RKHS $\mathcal{H}_K$ defined on the space of universal sets of items $\mathcal{S}$ with $|\mathcal{S}| = J$. Then, a kernel is defined as $\mathbf{K} : \mathcal{S} \times \mathcal{S} \to \mathbb{R}^{J\times J}$, which must satisfy symmetry and positive definiteness. The paper considers kernels for both featureless and feature-based settings as follow.

Featureless Setting

In featureless model, the entries of the kernel, also called the set kernel, depend only on the choice sets. A common definition is

$$[\mathbf{K}(S, S')]_{jk} = \mathsf{k}_{jk}(S, S') = \mathsf{k}_{jk}(\mathbf{e}_S, \mathbf{e}_{S'}), \tag{2.13}$$

where $\mathbf{e}_S \in \{0, 1\}^J$ represents the availability mask or the indicator for set S. To ensure $u_j(S) = 0$ if $j \notin S$, a masked set kernel $\mathbf{K}^S(S, S')$ is also defined with entries given by

$$\mathsf{k}^S_{jk}(S, S') = \mathbb{1}(j \in S) \cdot \mathbb{1}(k \in S') \cdot \mathsf{K}(S, S').$$
$$\mathbf{K}^S(S, S') = \mathrm{Diag}(\mathbf{e}_S)\mathbf{K}(S, S')\mathrm{Diag}(\mathbf{e}_{S'})$$

For example, [Yang et al., 2025] proposes the following polynomial kernel of order p to capture interaction (Halo) effects of up to p-th order:

$$\mathsf{k}^S_{j,k}(S, S') = \mathbb{1}(j \in S)\mathbb{1}(k \in S')(\boldsymbol{e}_S^\top \boldsymbol{e}_{S'})^p. \tag{2.14}$$

Feature-Based Setting

For scenarios with item features $\mathbf{X}_S \in \mathbb{R}^{d\times J}$, a product kernel is defined as

$$\bar{\mathbf{K}}((S, \mathbf{X}_S), (S', \mathbf{X}_{S'})) := \mathbf{K}^S(S, S') \cdot \mathbf{K}^{\mathcal{X}}(\mathbf{X}_S, \mathbf{X}_{S'}).$$

where $\mathbf{K}^S(S, S')$ is a masked ste kernel and $\mathbf{K}^{\mathcal{X}}(\mathbf{X}_S, \mathbf{X}_{S'})$ represents a feature kernel defined over items features.

A common choice for the feature kernel is a Gaussian kernel, specified by

$$[\mathbf{K}^{\mathcal{X}}(\mathbf{X}_S, \mathbf{X}_{S'})]_{ij} = \sigma^2 \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}'_j\|^2}{2\ell^2}\right),$$

where $\boldsymbol{x}_i$ is the i-th column of $\mathbf{X}_S$.

Now, consider a dataset $\mathcal{D}_N = \{(S_i, \mathbf{X}_{S_i}, \mathbf{y}_i)\}_{i=1}^N$ where S_i is the choice set, $\mathbf{X}_{S_i}$ is the feature matrix of the items in S_i, and $\mathbf{y}_i$ is the observed choice vector. Then, using a predefined kernel $\bar{\mathbf{K}}((S, \mathbf{X}_S), (S', \mathbf{X}_{S'}))$, the utility function can be formulated using the RKHS reproducing property as

$$\mathbf{u}(\cdot) = \sum_{i=1}^{N} \bar{\mathbf{K}}(\cdot, (S_i, \mathbf{X}_{S_i}))\boldsymbol{\alpha}_i \tag{2.15}$$

$$\mathbf{u}(S) = \langle \mathbf{u}(\cdot), \bar{\mathbf{K}}(\cdot, (S, \mathbf{X}_S)) \rangle \tag{2.16}$$

$$u_j(S) = \mathbf{e}_j^\top \sum_{i=1}^{N} \mathbf{K}^S(S, S_i)\mathbf{K}^{\mathcal{X}}(\mathbf{X}_S, \mathbf{X}_{S_i})\boldsymbol{\alpha}_i \tag{2.17}$$

where $\boldsymbol{\alpha}_i \in \mathbb{R}^J$ are the coefficients to be learned. The learning problem is then based on a regularized risk minimization

$$\min_{\mathbf{u} \in \mathcal{H}_{\bar{K}}} \frac{1}{N} \sum_{i=1}^{N} \ell(\mathbf{y}_i, \hat{\mathbf{y}}_i(\mathbf{u})) + \lambda \|\mathbf{u}\|_{\mathcal{H}^2_{\bar{K}}},$$

where $\hat{\mathbf{y}}_i(\mathbf{u})$ denotes the predicted choice probability vector for set S_i. This problem can finally be converted to a finite-dimensional convex problem over $\boldsymbol{\alpha}_i$s using the representer theorem.

Given this introduction on the RKHS method [Yang et al., 2025], we can now compare it with the DeepHalo model by Zhang et al. [2025]:

d.2 DeepHalo Method by Zhang et al. [2025]

Pros

- **Explicit interpretation of interaction order:** DeepHalo method can interpret interactions by order (pairwise, 3-way, etc.), as given in equation (2) of Zhang et al. [2025]. It clearly models how to handle maximum interaction order via depth L and quadratic activations (expressiveness up to order 2^{L-1}). It also provides criteria (e.g., relative context effect $\alpha_{jk}(T)$) to systematically identify and visualize item–item influences. Thus, it can explicitly identify common context effects such as decoy, similarity, and compromise effects.

- **Permutation-equivariance:** The DeepHalo method provides a permutation equivariant approach to model context effects. This equivariance supported by a theoretical proof (proposition 3.1 of Zhang et al. [2025]) makes the model more sample-efficient and robust to spurious pattern inference (e.g., always favoring the first column) and thus has a better inductive bias. This property is at least valid for

featured data settings based on equations (4) and (5) of Zhang et al. [2025]. Although in featureless setup, the aggregated reformulation in equation (10) is not permutation equivariant, one can still implement this model using a similar featured implementation with one-hot encoded features.

- **Scalable training:** Like most common neural architectures, DeepHalo training uses standard backpropagation and mini-batch stochastic gradient-based methods (e.g., Adam), which scales well to large sample sizes as long as the architecture is appropriately designed. It can also be easily integrated with GPU-accelerated frameworks for faster training.

Cons

- **Architectural complexity for high-order interactions:** The complexity of the DeepHalo method increases with the depth (number of layers). Using linear transformation for context effect modeling as in equation (4) of Zhang et al. [2025], one requires $L = |S| - 1$ layers to capture all interaction orders. The number of layers can be reduced with a polynomial activation function (for example, we require $L = \log_2(|S| - 1)$ with quadratic activation). For polynomials of high degrees, however, the optimization becomes unstable as the gradients begin to explode. As a result, the DeepHalo method may not efficiently model all interaction orders in choice sets with very large number of alternatives.

- **Non-convex training and optimization:** DeepHalo formulates a non-convex optimization problem due to non-convex NN activation functions (e.g., softmax). Thus, training can be sensitive to initialization, learning rates, and regularization strategies, and may converge to different local minima. There are also no strong theoretical guarantees of convergence or statistical learning bounds in Zhang et al. [2025].

- **Regularization and hyperparameter selection challenges:** DeepHalo requires careful selection of architectural hyperparameters (depth L, number of attention heads H, size of the embedding d, the nonlinear map and activation functions), regularization (weight decay, dropout), normalization (batch normalization layers) and optimization settings (algorithm, learning rate).

- **Limited feature integration:** DeepHalo, in its basic form, is limited to operate on item-level features, without integrating richer (shared) choice-set or individual-level covariates into the utility. Hence, the model cannot efficiently model market characteristics or person-specific behaviors.

d.3 RKHS Choice Model by Yang et al. [2025]

Pros

- **Convexity and global optimality:** The RKHS choice model leads to a convex optimization problem in the coefficients for a fixed kernel and convex loss (based on Theorem 1 of [Yang et al., 2025]), offering global optimality and greater stability in estimation and initialization.

- **Strong theoretical foundations:** Yang et al. [2025] provides strong theoretic guarantees which analyze both optimization and generalization behavior of the RKHS choice model. Theorem 2 shows that, in the neural regime, stochastic gradient descent on the corresponding neural implementation achieves an expected objective value that converges to the optimal RKHS risk at the classical $O(1/\sqrt{K})$ rate (with K being the number of iterations), despite the underlying nonconvex parameterization. Theorem 3 also provides a finite-sample generalization bound of order $O(1/\sqrt{N})$ for all utilities with bounded RKHS norm, controlling sample size and function complexity trade off in choice prediction.

- **Direct regularization control:** The RKHS model by Yang et al. [2025] offers a principled approach to prevent overfitting via regularization. It provides explicit norm-based regularization (e.g. $\|\mathbf{u}\|_{\mathcal{H}}^2$), enabling direct control of function smoothness and expressiveness with strong generalization bounds derived from RKHS theory.

- **Better feature integration:** In the RKHS method, each observation is $(S, \mathbf{X}_S)$, where columns of $\mathbf{X}_S$ are item features, and the kernel $\mathbf{K}^{\mathcal{X}}$ can be defined on any feature matrix. One can concatenate shared or individual-level features (e.g., market conditions, user attributes) to each column or add extra rows/columns encoding them. Alternatively, the product kernel can be reformulated by incorporating these shared features as:

$$\bar{\mathbf{K}}\left((S, \mathbf{X}_S, , \tilde{\mathbf{x}}_S), (S', \mathbf{X}_{S'}, \tilde{\mathbf{x}}_{S'})\right) = \mathbf{K}^{\mathcal{X}}(\mathbf{X}_S, \mathbf{X}_{S'})\mathbf{K}^{\tilde{\mathcal{X}}}(\tilde{\mathbf{x}}_S, \tilde{\mathbf{x}}_{S'})\mathbf{K}^{S}(S, S') \tag{2.18}$$

  where $\tilde{\mathbf{x}}_S$ denotes the shared choice/individual-level features. The RKHS method can consequently model utilities as functions of item features, choice-level attributes, and individual characteristics within a unified framework.

Cons

- **Not permutation-equivariant in the common sense:** The RKHS method by Yang et al. [2025] is not equivariant to the permutation of the items in the test

Table 2.1: Comparison of DeepHalo vs. RKHS Choice Model

Aspect	DeepHalo [Zhang et al., 2025]	RKHS Choice Model [Yang et al., 2025]
Permutation-equivariance	Yes, with symmetric aggregation and shared transformations	No, if considering within-set (test) permutation
Interpretability	High, pair-wise and higher order context effects (visualize item–item influences)	Moderate, global function properties, no submodular decomposition
Scalability	Scales well with large N via back propagation	Often $O(N^2)$ kernel matrix costs (could be reduced to $O(N)$ with random kernel approximation with the cost of losing precision)
Guarantees	Lacks a theoretical guarantee	Strong guarantees, generalization bounds, stability, convergence
Optimization	Non-convex, requires tuning, SGD	Convex, guarantees global optimum
Structural Requirements	Design network layers, activations, depth	Requires kernel selection, feature learning
Feature Integration	Weak: primarily works on item-level features	String: can incorporate individual/assortment-level features into kernel design
Application	Large datasets, need to measure specific context effects	Small/medium datasets, strong guarantees, stable optimization

set. That is because the kernel compares a test set $(S, \mathbf{X}_S)$ against fixed training sets $(S_i, \mathbf{X}_{S_i})$ using a matrix kernel $\bar{\mathbf{K}}$ on the ordered padded feature matrix $\mathbf{X}_S$. So changing the column (item) order as $\mathbf{X}_{\pi(S)} = \mathbf{X}_S\mathbf{P}$ (while keeping global item identities and training sets fixed) changes the kernel value and thus changes $u_j(\mathbf{X}_S)$

- **Poor scalability with large sample size N:** With fixed stationary kernels, pre-computation and storage of kernel values creates substantial memory challenge. The training complexity also scales as $O(J^2N^2)$ making RKHS method infeasible for massive datasets. The complexity can be reduced to $O(N_wNJ)$ with random kernel approximation as described in Section 3.3 of [Yang et al., 2025], where N_w is the number of discrete points used to approximate the distribution of the feature space. However, this efficiency gain comes at the cost of approximation error. It also requires learning a lower-dimensional random feature map as a secondary task.

- **Kernel selection is challenging and ad hoc:** A key disadvantage of the RKHS approach is that kernel selection and design (for both sets and features) is often challenging and somewhat ad hoc, requiring problem-specific choices and tuning and can strongly affect performance. This is evident in Fig. 1 of [Yang et al., 2025] which compares the performance of three different kernels including fixed, random features, and neural tangent kernels (NTK).

- **Limited interpretability of context effects:** The RKHS model by Yang et al. [2025] does not naturally yield a decomposition of the utility function into pairwise and higher-order interaction terms. In fact, in this method, interactions of different

orders are simultaneously and implicitly entangled inside the kernel expansion and the method does not provide direct, order-wise decomposition or explicit control over the highest interaction order. Hence, one cannot efficiently model specific context effects such as decoy or compromise effect.

e Credit Card Offer Demand Estimation and Key Challenges

Both RKHS choice model by [Yang et al., 2025] and DeepHalo model by [Zhang et al., 2025] can, in principle, be used for demand estimation of credit card offer. However, each comes with distinct advantages and disadvantages.

e.1 Application of RKHS Choice Model by Yang et al. [2025]

The RKHS-based choice model is a flexible nonparametric method that can capture complex, nonlinear relationships between consumer attributes, item features, and choice probabilities via appropriate kernel design. In a credit card offer setting, one can define kernels over:

- Cardholder features (spending history, demographics),
- Item-level features (merchant category, discount level),
- Offer set itself (assortment-level features such as maximum discount).

This approach lets the model capture diverse consumer behaviors and smooth changes in preference without relying on strict, predefined assumptions. This is especially useful in marketing and finance, where the true nature of demand is often unclear.

However, main challenges include:

- **Kernel design and engineering.** Choosing and tuning a suitable kernel (or combination of kernels) over high-dimensional, mixed-type features and potentially large offer sets is non-trivial. Poor kernel choice can limit the expressiveness of the RKHS method.
- **Scalability:** Standard RKHS methods involve kernel matrix operations that scale at least quadratically in the number of training examples; vector-valued kernels further increase computational cost. For millions of credit card users, one would likely need random feature approximation, introducing additional approximation error and design complexity.

- **Lack of permutation equivariance:** RKHS construction is not permutation equivariant with respect to reordering items within the test choice set. Practically, this means that the model can implicitly depend on how offers are indexed, rather than treating the menu as an abstract set, which is problematic when card offers change, new offers are introduced, or items are arranged in different orders across markets and time; in such settings, lack of built-in set symmetry can hurt robustness, complicate implementation, and make it harder to guarantee that predicted responses depend only on which offers and features are present, not on arbitrary positional encodings.
- **Context effects and high-order interactions.** While context dependence can be modeled via kernels on sets, interaction orders are not explicitly separated. If higher-order offer interactions (e.g., complex cannibalization/complementarity effects across many co-present offers) are important, the RKHS formulation may capture them implicitly but will not provide clean interpretability by interaction order.

e.2 Application of DeepHalo Model by Zhang et al. [2025]

DeepHalo is explicitly designed to handle context-dependent choice behavior, where choice probabilities depend not only on individual offers but also on the composition of the items in an offer set. This is directly relevant to credit card offers, where:

- Multiple offers are shown simultaneously to a customer,
- Certain offers may dominate or reinforce each other (e.g., overlapping merchant discounts),
- Behavioral effects such as attraction, compromise, and decoy effects may arise from the menu of offers.

However, there are also some challenges with the DeepHalo method:

- **Feature Integration Limit:** For the credit card offer problem, a central limitation of the DeepHalo model is its lack of native integration of shared choice-set or individual-level features. That is because the architecture is primarily designed over item-level features, rather than utilities that also depend on market- or person-specific features such as campaign type, or cardholder spending histories. While such information could be cast into the item feature vectors, this is heuristic and quickly becomes messy when the same shared variable applies to all offers in a choice. Hence, it is generally hard to capture realistic demand patterns where both offer attributes and consumer/market characteristics jointly affect response to promotions.

- **Complexity.** DeepHalo's ability to model higher-order interactions grows by adding more layers, but this also increases computational complexity and parameter count roughly with depth and number of items. This can become a disadvantage when estimating demand for credit card offers with many alternatives, as training and inference may become costly and harder to stabilize at scale. It may require large volumes of high-quality financial data with sufficient variation in observed assortments.

- **Non-convex optimization.** Training is non-convex and may be sensitive to initialization, architecture choice, and regularization. In production, this can translate into instability across repeated trainings and the need for careful design strategies to avoid overfitting (dropout, early stopping, normalization).

e.3 Summary

- The RKHS method [Yang et al., 2025] offers a theoretically-founded approach that is well suited to small- or medium-scale credit card offer demand estimation when the goal is to incorporate choice- and individual-level features for richer utility modeling; however, the lack of permutation equivariance means that item indexing and the way offers are aligned between training and test sets must be handled carefully to avoid artefactual biases in predictions.

- The DeepHalo method [Zhang et al., 2025] is particularly suitable for large datasets with millions of users, especially when context effects and higher-order interactions between offers are important and there is a premium on explicitly modeling and interpreting those interaction patterns. DeepHalo is more of a powerful but specialized context-effect module that would need to be embedded into a richer framework that incorporates shared assortment- or individual-level features.

Both methods still face several challenges for realistic credit-card offer demand estimation. For instance, neither model accounts for panel structure learning, habit formation, or stockpiling over time. These limitations will be addressed later in the bonus questions.

f Other (Non-Neural and ML) Approaches to Choice Modeling

Traditional discrete choice models beyond Yang's RKHS [Yang et al., 2025] and Zhang's DeepHalo [Zhang et al., 2025] and broader Machine Learning (ML) techniques may also be highly relevant for credit card offer demand estimation. These methods frequently

offer a more practical trade-off, balancing model complexity and predictive power while prioritizing essential real-world needs such as interpretability and robustness.

f.1 Other Discrete Choice Models

The **Multinomial Logit (MNL)** and **Conditional Logit** models [Train, 2009] are still important baselines in marketing and transportation. They are extremely fast to estimate even on very large datasets and yield directly interpretable coefficients (e.g., the marginal utility of price or the effect of specific merchant categories), which is valuable when the ultimate objective is to understand demand drivers and compute elasticities or willingness-to-pay (WTP) rather than just predict choices. Their main limitation is the independence-of-irrelevant-alternatives (IIA) property where they fail to capture context effects; however, as baseline methods, they are hard to beat in terms of simplicity and robustness.

To relax IIA while staying relatively simple, **Nested Logit** allows correlation in unobserved utility within nests of similar alternatives. For example, credit card offers can be grouped into nests such as "grocery", "travel", and "dining" rewards, with each nest having its own scale parameter [Train, 2009]. The nested logit probability can be written as a product of a conditional probability of choosing j within its nest and a probability of choosing the nest itself, both in logit form. This structure produces more realistic substitution patterns, with stronger interaction effects within nests (e.g., between two restaurant offers) than across nests, and is still estimated by relatively simple maximum-likelihood estimation methods. Compared to highly flexible models like DeepHalo, nested logit trades off some expressiveness for a much clearer behavioral interpretation of cross-substitution, which is crucial when designing or evaluating campaign policies.

Another important family is **Latent Class MNL** [Kamakura and Russell, 1989, Greene and Hensher, 2003]. Here the population is assumed to be a mixture of C latent segments, each with its own parameter vector $\boldsymbol{\beta}_c$, and each individual i belongs to class c with probability π_c. The class-conditional choice probabilities have the standard MNL form. This model captures taste heterogeneity in an interpretable way. For example, in credit card demand estimation, classes can be understood as segments of customers such as "reward maximizers" who are highly sensitive to price and discount, "brand loyal" customers who favor particular merchants, etc. It introduces discrete heterogeneity beyond a single MNL, is easy to estimate on large panels, and provides segment-level coefficients that directly support elasticity calculation and segment-specific offer design.

Halo MNL and its Low-Rank extension explicitly introduce context effects while retaining a logit-based structure. In Halo MNL, the utility of an item in an assortment includes not only its own attributes but also interaction terms capturing how the presence of other items (e.g., competing offers) alters its attractiveness, thereby modeling halo and

cannibalization effects in a parametric way. Low-Rank Halo MNL further constrains the interaction matrix to be low rank, which both reduces the number of free parameters and admits an efficient implementation, effectively interpolating between standard MNL (no interactions) and full Halo MNL (dense interactions). For credit card offer demand, these models can capture important context effects—such as a strong travel offer depressing response to weaker travel offers.

Taken together, these traditional models may be preferable or at least complementary to Yang's RKHS and Zhang's DeepHalo models in many practical settings: they are computationally lighter, have transparent behavioral structure, integrate naturally with tools for dealing with endogeneity (e.g., IV in BLP-type settings), and provide the kind of elasticity and WTP measures that credit card issuers need for pricing and campaign design.

f.2 ML Methods

ML techniques such as **support vector machines (SVM)**, **random forests**, and **gradient boosting** (e.g. XGBoost, LightGBM) may also be partially suitable for choice modeling: they can predict choices well. However, they do not, by themselves, provide the full structure that discrete choice analysis usually requires.

From a predictive viewpoint, suppose each observation is encoded as a feature *vector* $\mathbf{x}_i \in \mathcal{X}$ and a discrete (multi-class) choice label $y_i \in \{1, \ldots, J\}$. An SVM or gradient-boosted tree essentially learns a classifier function

$$f : \mathcal{X} \to \{1, \ldots, J\},$$

or, in probabilistic form,

$$p_\theta(y_i = j \mid \mathbf{x}_i) \approx \hat{p}_j(\mathbf{x}_i),$$

by minimizing an empirical loss (e.g. hinge loss or cross-entropy) plus regularization. In this sense, such models are fully capable of approximating

$$P(\text{alternative } j \text{ is chosen} \mid \mathbf{x}_i)$$

and often achieve high out-of-sample accuracy when $\mathbf{x}_i$ contains rich individual and alternative-specific features. Empirical benchmarks show that ensemble ML methods and deep learning can match or outperform classical logit-type models in pure prediction tasks [Püschel et al., 2024, Wang et al., 2024].

However, classical discrete choice models typically start from a random utility formulation,

$$u_{ij} = v_{ij} + \varepsilon_{ij}, \qquad P(y_i = j \mid \{v_{ik}\}_k) = \Pr\big(u_{ij} \geq u_{ik},\ \forall k\big),$$

where v_{ij} is a systematic utility with interpretable taste parameters (e.g. coefficients on price, time, and other attributes), and ε_{ij} has a specified distribution (logit, probit, etc.). This structure yields closed-form or numerically tractable choice probabilities, marginal effects, and elasticity measures derived from parameters such as β_{price}. Generic ML models do not impose this utility structure; they learn a flexible mapping $\mathbf{x}_i \mapsto \hat{p}(y_i \mid \mathbf{x}_i)$ without identified taste parameters. This makes it difficult to compute economically meaningful quantities like WTP in a transparent and stable way across counterfactuals.

Moreover, applied demand work often faces endogeneity (e.g. prices or offer targeting correlated with unobserved demand shocks). Identification strategies such as instrumental variables or control functions are naturally formulated in parametric discrete choice frameworks, whereas ML approaches such as SVM/forest/boosting models have no built-in mechanism to enforce the orthogonality conditions required for causal interpretation. Extending them to properly incorporate instruments is possible but nontrivial and not standard practice.

Finally, choice data have an intrinsic set structure: each observation involves a choice set S_i and alternative-specific attributes $\mathbf{x}_{ij}$ for $j \in S_i$. Classical models explicitly encode this via utility functions $v_{ij}(\mathbf{x}_{ij}, \mathbf{z}_i)$ and normalization across $j \in S_i$, ensuring coherent treatment of varying assortments and availability. Generic ML classifiers typically flatten the problem into a single vector $\mathbf{x}_i$, unless one explicitly engineers a representation that respects permutation equivariance and variable choice sets. Without such design, handling changing assortments (permutations) and availability in a principled way is difficult.

In summary, SVMs, random forests, and gradient boosting are suitable as powerful predictive models for discrete choices and useful as benchmarks or components, but they are not, by default, suitable as full replacements for discrete choice models when the goal is economic interpretation, causal demand estimation, or structured counterfactual analysis.

f.3 Experimental Evaluation

For a proven numerical evaluation, we implement different choice models on **SwissMetro** dataset from `choice_learn`. This dataset offers 3 transportation mode choices including TRAIN, SwissMetro, and CAR ($J = 3$), with item features such as availability, travel time, and cost, and shared features like purpose and age. For this experiment, we only include `'TT'` (travel time) and `'CO'` (cost) as the item features.

We compare our developed `DeepHaloChoiceModel` model with built-in choice models in `choice_learn.models` including `SimpleMNL`, `ConditionalLogit`, `LatentClassSimpleMNL`, `NestedLogit`, `HaloMNL`, and `LowRankHaloMNL`. We can build, train and evaluate these models using the methods and tools available in the `choice_learn`

package.

We then compare with the ML baseline algorithms such as SVM, Random Forest, and Gradient Boosting. The first two methods can be implemented using the `sklearn` package. For gradient boosting (LGB and XGB), we import `lightgbm` and `xgboost` packages.

For the ML methods, we prepare the training data in two distinct vector formats. First, we provide the plain features of each alternative in a choice set flattened as

$$\mathbf{x}_i = [a_{i1}\mathbf{x}_{i1}, a_{i2}\mathbf{x}_{i2}, \ldots, a_{iJ}\mathbf{x}_{iJ}]^\top, \quad y_i \in \{1, \ldots, J\}, \quad J = 3 \tag{2.19}$$

where $\mathbf{x}_{ij} = [\mathrm{CO}_{ij}, \mathrm{TT}_{ij}]$ represents cost and travel time for alternative j in choice situation i, and $a_{ij} \in \{0, 1\}$ encodes the availability of item j. Unavailable alternatives, indicated by the availability mask, are assigned zero feature vectors then. This approach may lead to a complex solution for high-dimensional item features. Another alternative is to use relative context features, such as the difference of each item's features with respect to the choice-set average and maximum values. The data vector then takes the form:

$$\mathbf{x}_i^j = [\mathbf{x}_{ij}, \mathbf{x}_{ij} - \bar{\mathbf{x}}_i, \mathbf{x}_{ij} - \max_k \mathbf{x}_{ik}, \frac{\mathbf{x}_{ij} - \bar{\mathbf{x}}_i}{\boldsymbol{\sigma}_i}], \quad y_i^j = \mathbb{1}(y_i = j) \tag{2.20}$$

where $\mathbf{x}_{ij}$ is the feature vector of item j in choice situation i, $\bar{\mathbf{x}}_i$ is the choice set mean, and $\boldsymbol{\sigma}_i$ is the standard deviation of the features in the choice set (across items). Specifically, we formulate the problem as a binary classification and treat each item in a choice set as a separate training sample. This increases the number of samples while keeping the dimensionality limited.

Table 2.2 shows the numerical results in terms of test accuracy and runtime (in seconds). Gradient-boosting methods (XGB and LGB) excel due to sequential error correction and built-in handling of class imbalance, interactions, and missing data. Contextual features also improve the results, especially for SVM and random forests (these models are specified by the "Context." prefix). These ML approaches are also generally faster than conventional choice-model algorithms. Nevertheless, the RUMnet choice model attains the highest accuracy and meets the economic-interpretability and demand-estimation requirements of choice modeling, despite its greater complexity.

Model	**Test Accuracy**	**Time (s)**
XGB	0.732	1.51
Context. XGB	0.702	0.79
LGB	0.697	0.96
Context. LGB	0.704	**0.65**
RandomForest	0.645	1.27
Context. RandomForest	0.693	2.98
SVM	0.645	10.91
Context. SVM	0.665	67.59
DeepHalo	0.667	30.71
SimpleMNL	0.617	4.59
ConditionalLogit	0.608	19.02
NestedLogit	0.678	14.24
HaloMNL	0.606	4.54
LowRankHaloMNL	0.606	40.99
RUMnet	**0.736**	303.06

Table 2.2: Test accuracy and runtime (in seconds) for classical discrete choice models and machine learning methods on SwissMetro dataset.

Chapter 3

Part 2: Discrete Choice Model with Sparse Market-Product Shocks

Introduction

Lu and Shimizu [2025] propose a new approach to estimating random-coefficient logit demand models for differentiated products assuming the vector of market-product-level demand shocks (unobserved characteristics) is sparse. The paper is motivated by the well-known Berry–Levinsohn–Pakes (BLP) framework [Berry et al., 1995], which incorporates unobserved product–market characteristics and uses instrumental variables (IV) to handle price endogeneity, but at the cost of demanding identification assumptions and computationally intensive demand inversion.

The key idea of Lu and Shimizu [2025] is to impose a structural sparsity assumption on the unobserved demand shocks and then use this structure, together with regularity conditions, to obtain nonparametric (deterministic) identification based on demand system of equations strategy that does not require IVs or demand inversion. The authors then develop a Bayesian shrinkage procedure to estimate both preference parameters and the sparse vector of shocks, and compare its performance to BLP-type estimators in simulations and empirical applications.

In a random-coefficient logit model, the utility of consumer i for product j in market t is modeled as

$$u_{ijt} = \mathbf{x}_{jt}^{\top}\boldsymbol{\beta}_i + \xi_{jt} + \varepsilon_{ijt},$$

where $\mathbf{x}_{jt}$ are observed characteristics, $\boldsymbol{\beta}_i$ are heterogeneous tastes (which vary between individuals) modeled with random variables, ξ_{jt} is the market–product-level demand shock and ε_{ijt} is an i.i.d. (idiosyncratic) error.

The demand shock ξ_{jt} captures unobserved characteristics of product j in market t that are known to firms and consumers but not to the econometrician.

A concrete example of a market–product shock ξ_{jt}: In week t, a supermarket runs

an unadvertised in-store campaign for Brand A yogurt: better shelf placement, in-aisle sampling, and a special end-cap display. Regular shoppers and the retailer know this; it clearly boosts Brand A's appeal that week. But in your dataset you only observe price, flavor, size, and a coarse "on promotion" dummy. You do not observe the exact shelf placement quality or sampling effort. In the BLP framework, these shocks are the main source of price endogeneity, because firms condition prices on ξ_{jt} whereas the researcher does not observe them.

Lu and Shimizu [2025] assume that in some markets, the vector of shocks $\boldsymbol{\xi}_t = (\xi_{1t}, \ldots, \xi_{J_t t})$ is sparse in the sense that many products share a common market-level component and only a relatively small subset of products has distinct deviations from this common value.

a Errors and Unclear Parts in [Lu and Shimizu, 2025]

In this section, we identify and explain potential errors, ambiguities, and unclear parts in [Lu and Shimizu, 2025]. These errors are listed as follows:

- The notation $(s_{0t}, \ldots, s_{J_t t}) \in \Delta^{J_t}$ is incorrect. The market share vector should be a member of Δ^{J_t+1} (considering the outside option).

- It is not clear how maximization of the (Maximum Likelihood) ML function in eq. (3) relates to solving the demand system in eq. (4). In fact, the sentence "... estimating $(f, \boldsymbol{\xi}_1, \ldots, \boldsymbol{\xi}_T)$ based on likelihood function (3) ..." is incorrect.

- Assumption 1 is based on an infinite population of markets and products, but practical applications involve finite samples. For example, in the yogurt application ($T = 95$ stores-weeks, $J_t \approx 62$ products), neither the number of markets nor products is infinite. The paper does not provide a finite-sample analogue of Assumption 1 and does not justify the applicability of their method on finite samples. In other words, the paper does not specify conditions on $|\mathcal{S}|/T$ and $|\mathcal{K}_t|/J_t$ under which identification approximately holds in practice.

- The independence assumption for $\mathbf{x}_{jt}$ across j within a market t is unrealistic. Prices within a market/week are correlated due to common market conditions. Other product characteristics (brand, size, etc.) are also often related and clustered across products. Market-level factors (seasonality, competition) induce dependence across products in the same market-time frame. Furthermore, the product characteristics in some applications may not be continuous (e.g., the quantities). Hence, Assumption 2 may not hold in practice.

- The identification proof relies on full support and independence of $\mathbf{x}_{jt}$ to obtain uniqueness of the Laplace transform, but the exact mechanism by which independence guarantees identification is not clearly specified.

- The notation for the set $\mathcal{K}_t = \{K_t + 1, \ldots, J_t\}$ is confusing, since one may implicate that $|\mathcal{K}_t| = K_t$, while this is not true. It is also suggested to use lowercase letter for the set cardinaloty as k_t.

- In Theorem 1, the condition $t = 1, \ldots, T$, should be changed to $t \in \mathcal{S}$ where $\mathcal{S}$ is defined in Assumption 1.

- The notation used for the standard normal density $\phi(\cdot|\mathbf{0}, \mathbf{I})$ should be changed as it may be confused with the inclusion probability parameter ϕ_t.

- The notation $\dim(f)$ is used in a nonstandard way. Typically, "dimension" refers to d_X, the dimension of $\boldsymbol{\beta}$, not the number of free parameters of f.

- The exponential (or subexponential) tail restriction on f lacks intuitive justification. The paper does not explain why such a tail condition is specifically needed for the Laplace-transform-based identification argument. In other words, there is no proof of why an exponential distribution satisfies the condition in Assumption 3. There is also no discussion of what happens when f has heavier tails (e.g. Student-t); it is unclear whether identification fails or the assumption is stronger than necessary.

- In Assumption 3, the existence of the Laplace transform is examined over bounded balls B in $\mathbf{x}$-space (Laplace frequency domain), whereas Laplace-transform uniqueness usually involves behavior on unbounded domains; the motivation for restricting to bounded B is not made explicit.

- The uniqueness argument for the subsystem solution in eq. (8) is incomplete. In fact, there is no exact proof that the additional $J_t - (K_t + 1)$ equations are consistent with the first $K_t + 1$ equations.

- Theorem 1 assumes the sparse structure Ξ_t is known, but in practice the sets $\mathcal{S}$ and $\{\mathcal{K}_t\}_{t \in \mathcal{S}}$ are latent; the identification of the sparsity pattern itself is not addressed. Additionally, there is a disconnect between the theorem (which conditions on a known sparsity structure) and the estimation section, which uses spike-and-slab priors to learn γ_{jt} from the data without a formal identifiability result for the support. In simple words, the paper provides guaranties for a non-parametric identification, while the proposed Bayesian estimation method applies to a parametric model.

- The proof for uniqueness of solutions in Theorem 1, does not depend on K_t (or the degree of sparsity). In other words, the market can be even completely non-sparse ($K_t = J_t - 1$) and the proof still hold. Hence, it seems that the proof has missing conditions on the cardinality of $\mathcal{K}_t$.

- In equation (20) in Appendix A.2 of [Lu and Shimizu, 2025], Assumption 3 is utilized to prove that the integral is finite and the transform is well defined. However, this requires $\exp\left(H_t(\boldsymbol{\beta}, f)\right) > 1$ or equivalently $H_t(\boldsymbol{\beta}, f) > 0$ which does not necessarily hold. For example, if $\nu_t = \tilde{\sigma}^{-1}_{K_t+1,t} \to \infty$, then we must have $\sum_{k=K_t+1}^{J_t} \exp(\mathbf{x}_{kt}^\top \boldsymbol{\beta}) > 1$ which may not hold in general.

- In Appendix B.1, the likelihood given by the observed market shares $\mathbf{q}$ is repeatedly mistaken for the posterior probability. The term $p(\bar{\boldsymbol{\beta}}, \bar{\boldsymbol{\xi}}, \boldsymbol{\eta}, \mathbf{r}|\mathbf{q})$, (incorrectly) referred to as likelihood, should be replaced by $p(\mathbf{q}|\bar{\boldsymbol{\beta}}, \bar{\boldsymbol{\xi}}, \boldsymbol{\eta}, \mathbf{r})$.

b Replication of Results in Section 4 of [Lu and Shimizu, 2025]

b.1 Our Implementation

Here, we explain our implementation of the Lu and Shimizu [2025] method in choice learn. Our code is an MCMC sampler for an aggregate-data random coefficient logit with sparse shocks. It closely mirrors the computation details in the appendix of [Lu and Shimizu, 2025]. We implement block Random-Walk Metropolis-Hastings (RWMH) updates for $\bar{\boldsymbol{\beta}}$, logarithmic standard deviation values $\mathbf{r}$, the inclusion probabilities ϕ_t (RWMH update in the logit domain), and market intercepts $\bar{\xi}_t$, and the sparse shocks η_{jt} with spike-and-slab-dependent step sizes. This is to avoid very low acceptance rates in spike mode. We also perform Gibbs updates for γ_{jt} (binary inclusion or indicator variables).

We implement the MCMC samplers using `tfp.mcmcm`[1] module. To sample from ϕ_t based on RWMH, we first define a logit mapping $\text{logit}(\phi_t) : [0, 1] \to \mathbb{R}$. We then propose RWMH updates in the logit domain. Let this mapping be represented as

$$z_{\phi_t} = \text{logit}(\phi_t) = \log \frac{\phi_t}{1 - \phi_t}, \quad \text{so } \phi_t = \sigma(z_{\phi_t}).$$

Then, we propose the next state as:

$$z'_{\phi_t} = z_{\phi_t} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \text{step}^2)$$

[1] https://www.tensorflow.org/probability/api_docs/python/tfp/mcmc

We then compute the log-posterior of z_{ϕ_t} via the change-of-variables formula:

$$\log p(z_{\phi_t} \mid \gamma) = \log p(\phi_t \mid \gamma_t) + \underbrace{\log \phi_t + \log(1-\phi_t)}_{\text{log Jacobian of } \sigma(\cdot)}$$

where $\log p(\phi_t \mid \gamma_t)$ is the Beta log-probability:

$$\log p(\phi_t \mid \gamma_t) = \log \text{Beta}\left(\phi_t;\, a_\phi + \sum_j \gamma_{tj},\, b_\phi + J - \sum_j \gamma_{tj}\right)$$

We finally, accept/reject with the standard MH ratio:

$$\alpha = \min\left(1,\, \frac{p(z'_\phi)}{p(z_\phi)}\right)$$

To sample from γ_{jt}, we employ a custom Gibbs kernel realized by sub-classing the `TransitionKernel` in `tfp.mcmc` module. The reason for this is that, γ_{jt} is a binary variable and unlike the other parameters, we cannot simply propose continuous RWMH updates. Hence, we implement the original Gibbs sampler as in [Lu and Shimizu, 2025] using a custom transition kernel. For this purpose, we need to compute the conditional log-posterior of $\boldsymbol{\gamma}$ given the other variables as

$$\log p(\boldsymbol{\gamma} \mid \boldsymbol{\eta}, \boldsymbol{\phi}) = \sum_{t,j}\Big[\gamma_{tj}\log\phi_t + (1-\gamma_{tj})\log(1-\phi_t)\Big] + \sum_{t,j}\Big[-\frac{\eta_{tj}^2}{2\,\text{var}_{tj}} - \frac{1}{2}\log \text{var}_{tj}\Big]$$

where

$$\text{var}_{tj} = \gamma_{tj}\tau_1^2 + (1-\gamma_{tj})\tau_0^2.$$

This leads to the exact formulation for $\Pr(\gamma_{jt}=1 \mid \cdot)$ as in [Lu and Shimizu, 2025]:

$$\Pr(\gamma_{jt}=1 \mid \cdot) = \sigma\big(\log p(\gamma^{(1)}) - \log p(\gamma^{(0)})\big)\,,$$

where $\sigma(\cdot)$ is the logistic sigmoid and $\gamma^{(1)}$, $\gamma^{(0)}$ differ only at site (t,j). Then, we simply sample from γ_{tj} by generating a uniform number $u_{jt} \sim U[0,1]$ and setting (with 100% acceptance)

$$\gamma_{tj} = \mathbb{1}\left(u_{jt} < \Pr(\gamma_{jt}=1 \mid \cdot)\right).$$

We also do a block-update for the sparse shocks η_{jt} with a proposal scale (step size) proportional to variance of the spike/slab regime.

Further details about our code and its constituting methods and elements are as follows:

Parameter classes and hyperparameters

`SimParams`

Controls the DGP: (T, J, N_t, D), the mean and std of random coefficients, the baseline shock level `xi_bar`, and a random seed. The `sigma_beta` vector supports fixed coefficients by setting a component to 0.

`MCMCParams`

Holds algorithmic and prior hyperparameters:

- Monte Carlo integration: `R0` simulated draws for random coefficients.
- Chain length: `G` iterations and `burn` burn-in (used for adaptation).
- Random vs fixed coefficients: `random_coef_mask` in $\{0,1\}^D$ and derived index sets `random_indices`, `fixed_indices`.
- Spike-and-slab prior scales: `tau0` (spike std) and `tau1` (slab std), corresponding to the paper's small vs large prior variance components.
- Inclusion probability hyperparameters: `a_phi`, `b_phi` for $\phi_t \sim \text{Beta}(a_\phi, b_\phi)$, mapping to the paper's Beta prior on market-level inclusion probability (denoted π_t in the paper).
- Gaussian priors for continuous parameters: `V_beta`, `V_xibar`, `V_r` act as prior variances for $\bar{\boldsymbol{\beta}}$, ξ_t, and r respectively (normal priors centered at 0).
- Proposal scales for RWMH updates: `step_beta`, `step_r`, `step_xibar`, `step_eta`, `step_phi`.
- Target acceptance rate for is used by `tfp.mcmc.SimpleStepSizeAdaptation`.

Synthetic data generation

The `GenerateData.py` module contains the generator class `generate_data_tf` that creates:

- T markets, J inside goods, plus outside option ($J+1$ alternatives total).
- D observed item features per product (e.g. price and a characteristic).
- Consumer-level taste draws $\epsilon_{it} \in \mathbb{R}^D$ that generate individual $\boldsymbol{\beta}_i \in \mathbb{R}^D$ around `beta_mean` with std `sigma_beta`.
- A sparse shock pattern $\eta^\star_{tj}$ for the market–product deviations and $\xi^\star_{tj} = \bar{\xi}_t + \eta^\star_{tj}$.

Having this, the utilities are simulated as

$$u_{itj} = \boldsymbol{\beta}_i^\top \mathbf{x}_{tj} + \xi_{tj}^\star,$$

with the outside option utility set to 0. Probabilities are computed by log-sum-exp, then aggregate counts q_{tj} are formed by rounding $N_t \cdot s_{tj}$. This yields an aggregate dataset $(\mathbf{X}, \mathbf{q}, N_t)$ for the MCMC likelihood and also an individual-level `ChoiceDataset` by expanding counts into repeated choices for compatibility with the `choice_learn` API.

Although the `ChoiceDataset` is stored as one row per (expanded) individual choice, the sampler does not run MCMC at the individual level. Instead it constructs a unique-market representation:

- `unique_counts`: a tensor of shape $(T_{\text{unique}}, J+1)$ containing counts q_{tj}.
- `unique_items_features`: a tensor of shape $(T_{\text{unique}}, J+1, D)$ containing item features $\mathbf{x}_{tj}$, including a prepended outside option row of zeros.

It should be noted that in our implementation, market identifiers are stored inside the `ChoiceDataset` as a shared feature per expanded observation, and later recovered from that shared-feature array when the Bayesian sampler re-aggregates the data to unique markets. Specifically, the first column of the shared features matrix is assumed to contain the market IDs which we use to subsequently find the `unique_market_ids` and `unique_counts`.

Bayesian Sparse ChoiceModel

The `LuSparseRandomLogit.py` module contains the `BayesianSparseRandomLogit` class that wraps the Bayesian sparse choice model of [Lu and Shimizu, 2025] with `choice_learn` package. Our model is built upon `ChoiceModel` which features the `compute_batch_utility` and `fit` methods (for interoperability with other `choice_learn` routines). However, the actual estimation is not gradient descent; it is MCMC with explicit RWMH and Gibbs transitions. Therefore, `trainable_weights` is provided only to satisfy the API and returns the current state variables rather than parameters being optimized.

After initialization, the model maintains the following current MCMC state:

- `beta_bar` $\in \mathbb{R}^D$: mean coefficients $\bar{\boldsymbol{\beta}}$.
- `n_random` The number of random elements in $\bar{\boldsymbol{\beta}}$
- `r_vec` $\in \mathbb{R}^{n_{\text{random}}}$: log standard deviations for the random coefficient components, with $\boldsymbol{\sigma} = \exp(\mathbf{r})$. This matches the paper's reparameterization $r_k = \log \sigma_k$.
- `xi_bar` $\in \mathbb{R}^{T_{\text{unique}}}$: market-level shocks ξ_t.

- `eta` $\in \mathbb{R}^{T_{\text{unique}} \times J}$: product deviations η_{tj} for inside goods only (outside option excluded).
- `gamma` $\in \{0,1\}^{T_{\text{unique}} \times J}$: spike-and-slab indicators γ_{tj} for each market–product deviation.
- `phi` $\in (0,1)^{T_{\text{unique}}}$: market-level inclusion probabilities ϕ_t controlling the Bernoulli prior/probability of $\gamma_{tj} = 1$.
- `v_draws` $\in \mathbb{R}^{R_0 \times n_{\text{random}}}$: fixed simulation draws used to approximate integrals over random coefficients.

The following functions (methods) are defined in this module:

`compute_batch_utility:` This is used for per-choice utilities in the `choice_learn` style. It:

1. looks up the market id for each choice and gathers $\bar{\xi}_t$ and η_{tj},
2. computes a deterministic part from $\bar{\boldsymbol{\beta}}^\top \mathbf{x}_{tj}$,
3. adds $\xi_{tj} = \bar{\xi}_t + \eta_{tj}$ (with 0 for the outside option),
4. applies availability masks by setting utility to a large negative number for unavailable items.

Note: in this function, random coefficients are not integrated; it uses $\bar{\boldsymbol{\beta}}$ only. It is just a `choice-learn` API-compatible utility map, not the sampler's likelihood evaluation.

`compute_batch_utility_unique:` This is the core for the aggregate likelihood:

- For fixed coefficients d (indices in `fixed_indices`), it adds $\beta_d x_{tjd}$ directly.
- For random coefficients d' (indices in `random_indices`), it forms R_0 coefficient draws as
$$\beta_{d'}^{(r)} = \bar{\beta}_{d'} + \exp(r_{d'})\, v_{rd'},$$
then adds $\beta_{d'}^{(r)} x_{tjd'}$ to the utility.
- It produces utilities of shape $(T_{\text{unique}}, R_0, J+1)$.
- It then adds ξ_{tj} (padded with outside option 0).
- Again availability mask is applied to mask out the utility for unavailable items.

This corresponds to the paper's simulated share formula where the integral over heterogeneity is approximated with R_0 draws.

`compute_log_likelihood_unique:` Given inside-good shocks `xi` of shape (T_{unique}, J), it computes:

1. utilities u_{rtj},
2. choice probabilities p_{rtj} via softmax,
3. simulated shares $\sigma_{tj} = \frac{1}{R_0} \sum_{r=1}^{R_0} p_{rtj}$,
4. market log-likelihood contributions

$$\ell_t = \sum_{j=0}^{J} q_{tj} \log(\sigma_{tj}).$$

`adapt_step_size` A generic helper that updates one scalar step size given acceptance bounds and is used only during a pilot phase (after burn-in windows).

b.2 Results

This part details the computational implementation of the Bayesian shrinkage algorithm for random coefficients logit model in [Lu and Shimizu, 2025] with sparse shocks. We particularly replicate the simulation results in Section 4 of [Lu and Shimizu, 2025]. We compare the performance of the Bayesian shrinkage method of Lu and Shimizu [2025] with the BLP model of Berry et al. [1995], which is based on the proper identification of the instrumental variables (IV). Three variants of BLP are used as benchmarks, where the first two are denoted by BLP (with cost IV) and BLP (without cost IV). We also propose a third variant which exploits exogenous features of the competitor products to build the IVs.

The results are provided in Tables 3.1 to 3.7. In these tables, the bias/SD values of $\boldsymbol{\xi}$ are the averages of (absolute value of) bias/SD of ξ_{jt}. The "Int" quantity also represents the intercept term $\bar{\xi}_t$. Due to limitations in computational and memory resources, we have run the experiments over 10 repetitions of the DGP data generation samples and averaged the results. Figure 3.4 presents the output from one repetition of the MCMC samples and their corresponding histograms for DGP1. For the Bayesian shrinkage method of [Lu and Shimizu, 2025], we plot the trace and histograms of the MCMC samples for $\bar{\beta}_0$, $\bar{\beta}_1$, σ_0, $\bar{\xi}_t$ (averaged over all markets), and η_{ij} (averaged over all markets and products). We additionally plot the trace of ϕ_t (averaged over all markets) along with its per-market sample mean, as well as the trace of γ_{jt} (averaged over T and J) with a histogram of the sample means of all its elements. We also plot the estimates obtained using different variants of the BLP for $\bar{\beta}_0$, $\bar{\beta}_1$, σ_0, and. $\bar{\xi}_t$. In these plots, BLP1 represents the strong BLP (with cost IV), BLP2 represents the weak BLP (without cost IV), and BLP3 represents our proposed BLP based on competitor characteristics.

These results closely match those reported in Section 4 of [Lu and Shimizu, 2025]. First, as expected, the Bayesian shrinkage method outperforms the benchmark BLP estimators in the presence of sparse market-product shocks (DGP1 and DGP2). However, for the non-sparse data generation process with an exogenous price variable, the strong BLP (with cost shock) appears to be the best. Yet, when price endogeneity is introduced, as in DGP4, Lu's Bayesian shrinkage method again emerges as the top performer.

BLP1 with cost IV (strong BLP) is expectedly shown to perform better than the BLP2 variant that does not exploit cost IV. The third variant of BLP (BLP3), based on competitor features, appears not to be efficient in estimating the market intercept shock Int and the mean taste parameters $\bar{\boldsymbol{\beta}}$, especially for DGP1-DGP3. However, it always estimates the variance of the taste parameters σ with almost zero error, outperforming every other method in this sense.

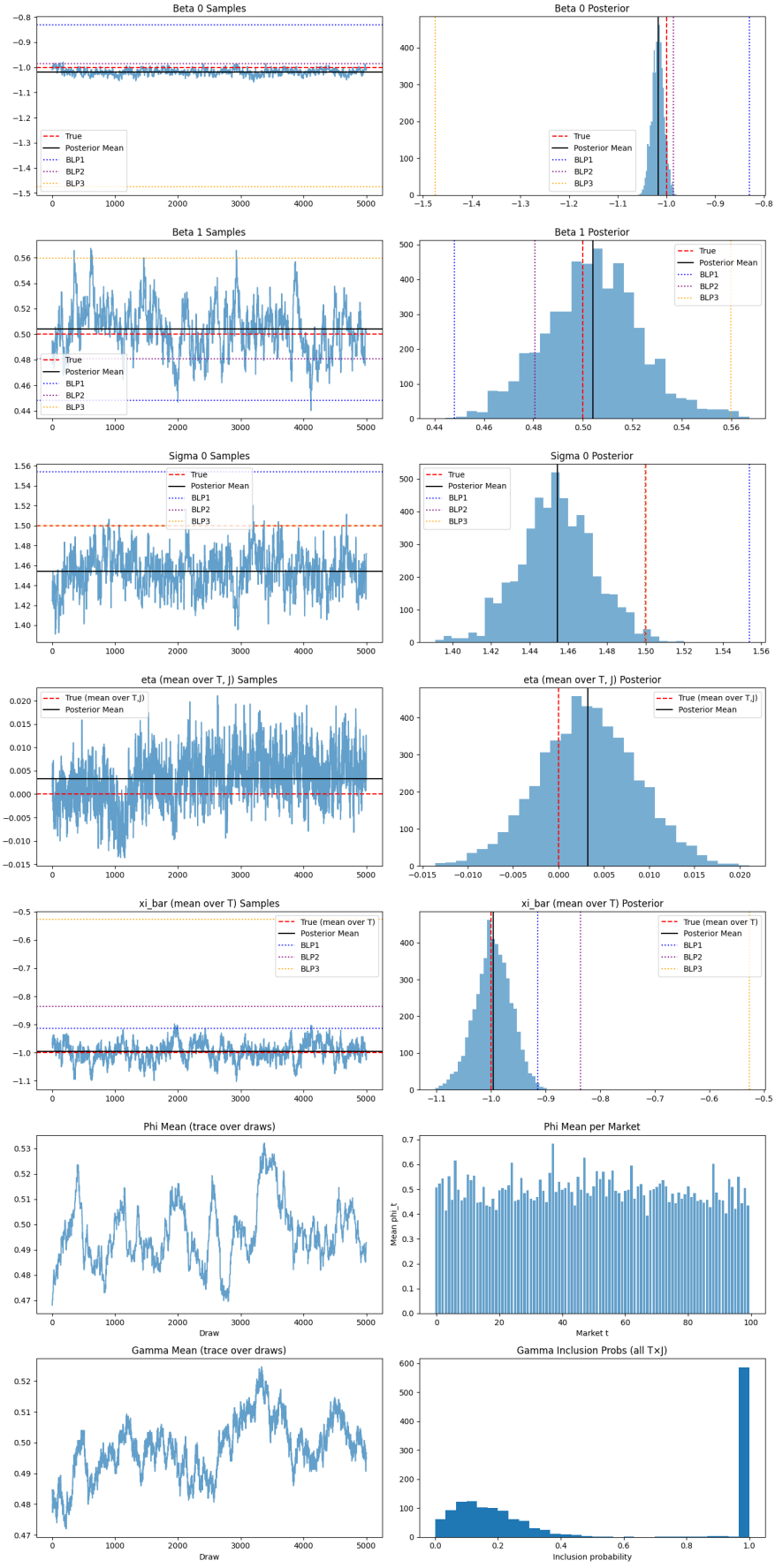


Figure 3.1: One repetition output of the MCMC samples from the Lu's Bayesian method compared to the BLP estimates for DGP1 data. Here BLP1 represent the strong (with cost IV) BLP, BLP2 represents the weak-IV (without cost) BLP, and BLP (no cost/ competitors) is our proposed BLP3 with sum of the features of the competitor products as IVs.

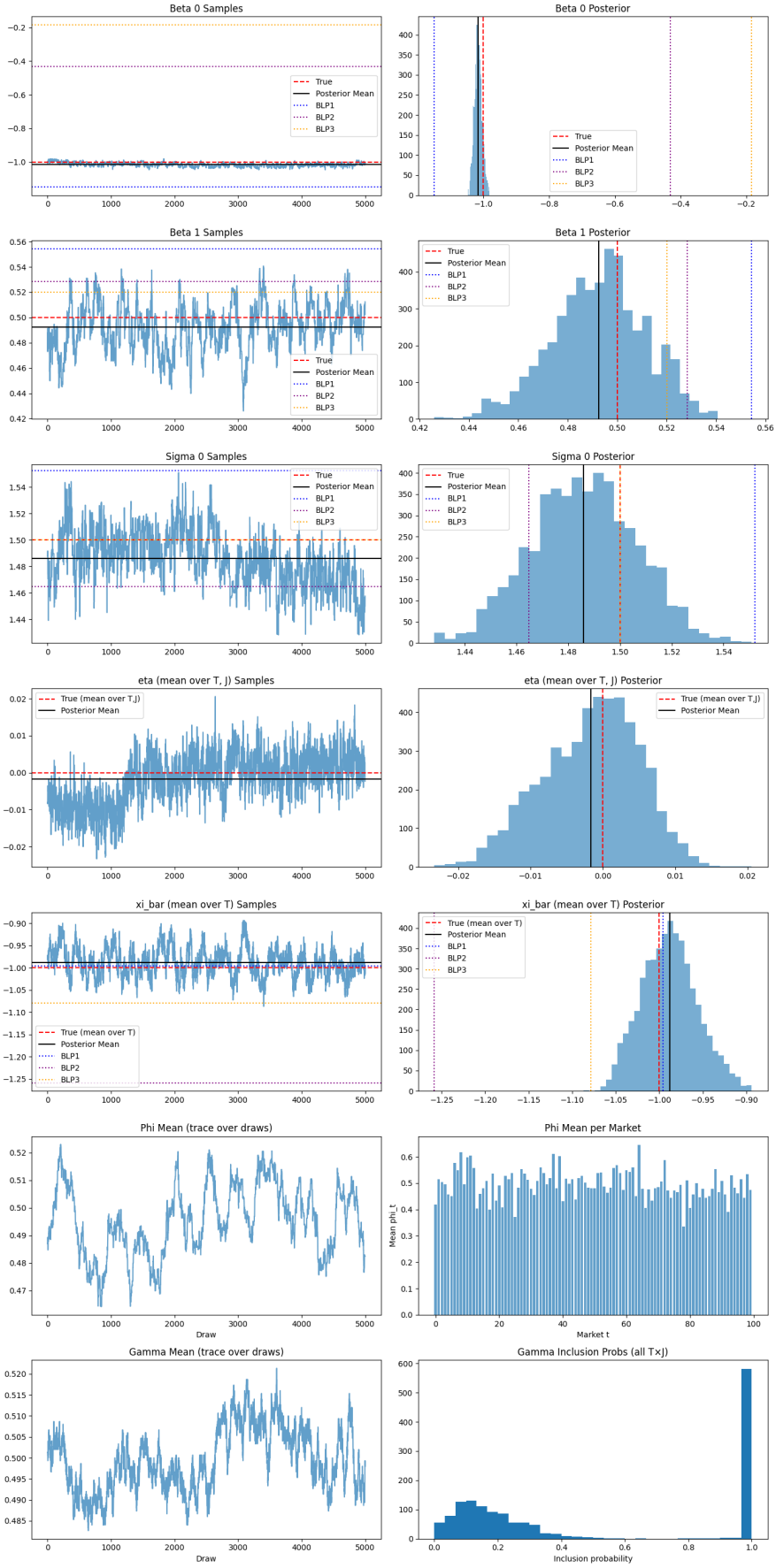


Figure 3.2: One repetition output of the MCMC samples from the Lu's Bayesian method compared to the BLP estimates for DGP2 data. Here BLP1 represent the strong (with cost IV) BLP, BLP2 represents the weak-IV (without cost) BLP, and BLP3 (no cost/ competitors) is our proposed BLP with sum of the features of the competitor products as IVs.

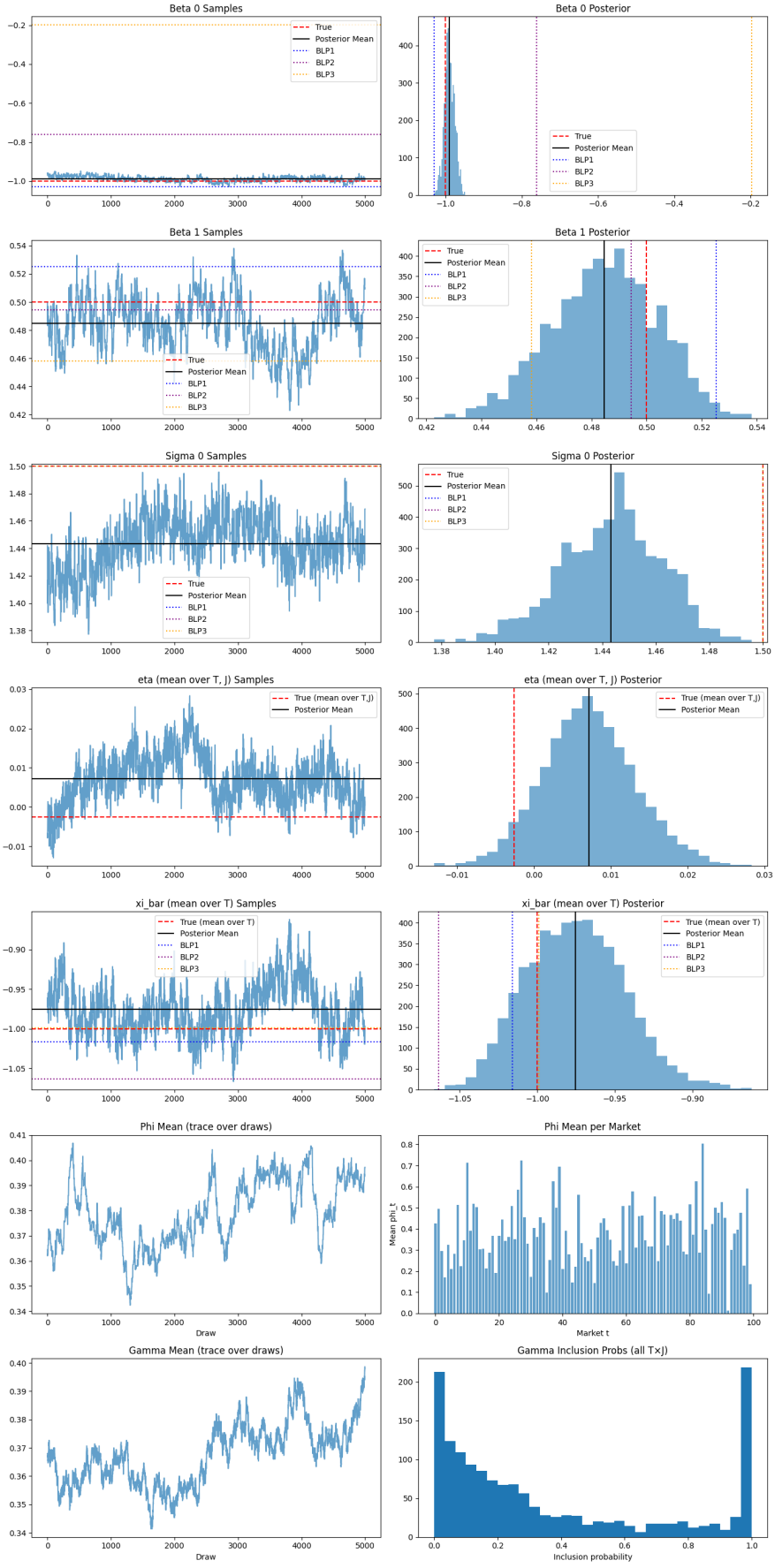


Figure 3.3: One repetition output of the MCMC samples from the Lu's Bayesian method compared to the BLP estimates for DGP3 data. Here BLP1 represent the strong (with cost IV) BLP, BLP2 represents the weak-IV (without cost) BLP, and BLP3 (no cost/ competitors) is our proposed BLP with sum of the features of the competitor products as IVs.

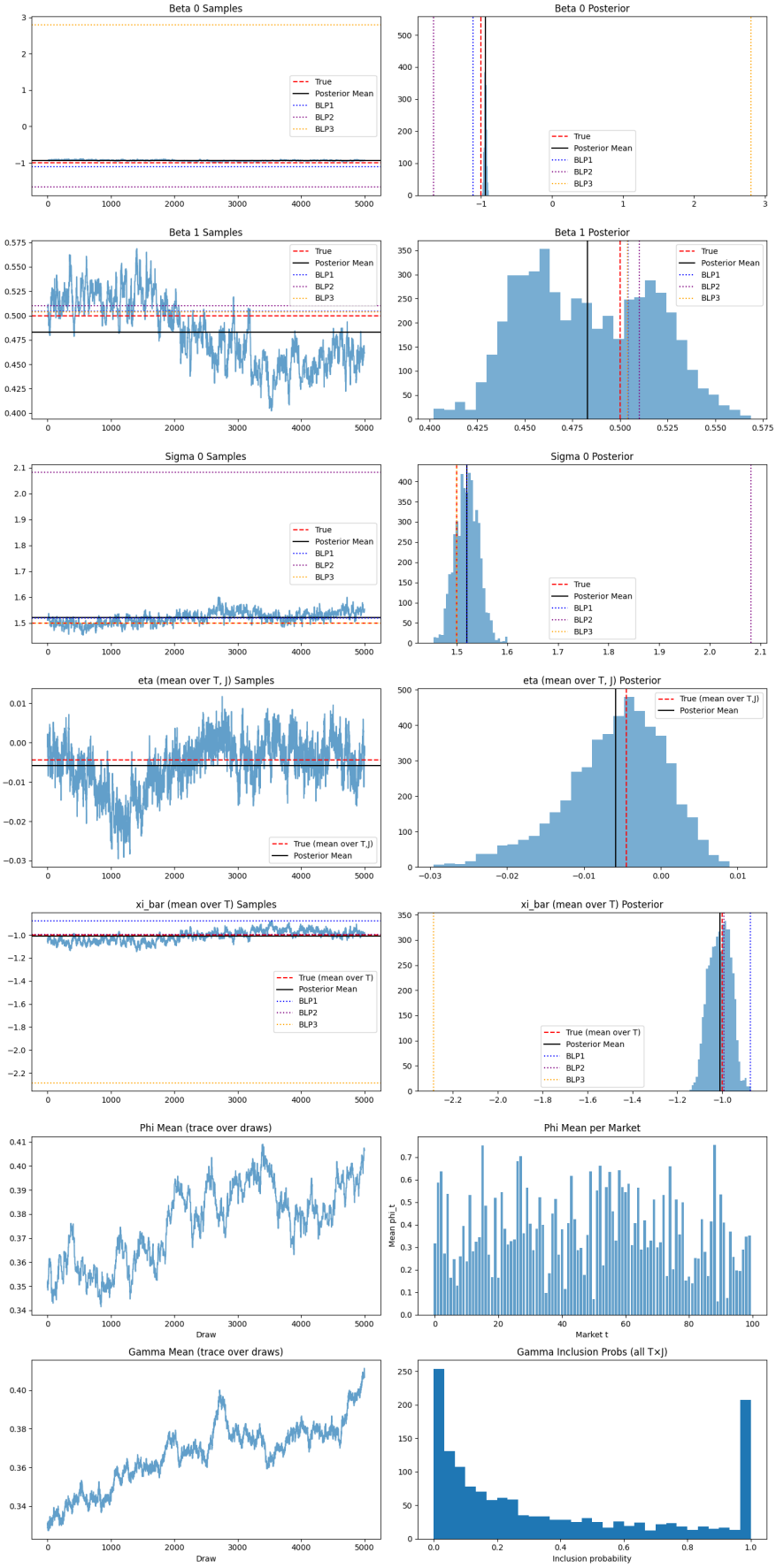


Figure 3.4: One repetition output of the MCMC samples from the Lu's Bayesian method compared to the BLP estimates for DGP4 data. Here BLP1 represent the strong (with cost IV) BLP, BLP2 represents the weak-IV (without cost) BLP, and BLP3 (no cost/ competitors) is our proposed BLP with sum of the features of the competitor products as IVs.

Table 3.1: Simulation results of DGP1 (sparse exogeneous case)

			BLP (with cost IV)					BLP (without cost IV)					Shrinkage				
J	T		Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ
5	25	Bias	0.11	-0.03	-0.08	0.11	0.11	0.12	0.05	-0.10	0.11	0.15	0.02	-0.01	-0.01	-0.02	0.03
		SD	0.36	0.14	0.24	0.16	0.38	0.63	0.62	0.31	0.61	0.76	0.11	0.05	0.07	0.13	0.12
5	100	Bias	-0.04	-0.04	0.02	0.13	0.04	0.12	-0.21	0.00	-0.12	0.14	-0.00	-0.02	0.00	-0.04	0.01
		SD	0.17	0.12	0.11	0.15	0.19	0.27	0.62	0.16	0.26	0.48	0.03	0.01	0.02	0.01	0.05
15	25	Bias	0.06	0.03	-0.01	0.04	0.06	0.12	-0.03	-0.06	0.11	0.17	0.00	-0.01	0.00	-0.04	0.02
		SD	0.18	0.11	0.11	0.09	0.19	0.34	0.71	0.16	0.78	0.59	0.03	0.02	0.02	0.02	0.05
15	100	Bias	0.08	-0.00	-0.03	0.01	0.08	0.01	-0.15	-0.00	0.28	0.19	0.02	-0.01	-0.01	-0.05	0.03
		SD	0.08	0.09	0.05	0.02	0.11	0.43	1.12	0.07	0.49	0.79	0.02	0.01	0.01	0.01	0.04

(a) Comparison between strong BLP (with cost IV), weak BLP (without cost shock), and the Shrinkage method of [Lu and Shimizu, 2025].

			BLP (with cost IV)					BLP (without cost IV)					BLP (no cost/ competitors)				
J	T		Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ
5	25	Bias	0.11	-0.03	-0.08	0.11	0.11	0.12	0.05	-0.10	0.11	0.15	-0.89	1.27	0.19	0.00	1.21
		SD	0.36	0.14	0.24	0.16	0.38	0.63	0.62	0.31	0.61	0.76	4.62	5.61	1.28	0.00	5.22
5	100	Bias	-0.04	-0.04	0.02	0.13	0.04	0.12	-0.21	0.00	-0.12	0.14	0.81	-2.44	0.25	0.00	1.12
		SD	0.17	0.12	0.11	0.15	0.19	0.27	0.62	0.16	0.26	0.48	1.81	4.67	0.39	0.00	3.73
15	25	Bias	0.06	0.03	-0.01	0.04	0.06	0.12	-0.03	-0.06	0.11	0.17	0.07	0.91	-0.11	0.00	0.55
		SD	0.18	0.11	0.11	0.09	0.19	0.34	0.71	0.16	0.78	0.59	1.00	3.17	0.28	0.00	2.01
15	100	Bias	0.08	-0.00	-0.03	0.01	0.08	0.01	-0.15	-0.00	0.28	0.19	0.44	-0.42	0.03	0.00	0.49
		SD	0.08	0.09	0.05	0.02	0.11	0.43	1.12	0.07	0.49	0.79	0.75	1.82	0.06	0.00	1.37

(b) Comparison between Lu's BLP benchmarks and our proposed BLP variant without cost (competitor features).

Note: The bias/SD of ξ are the averages of (absolute value of) bias/SD of ξ_{jt}. Int = the intercept term $\bar{\xi}$.

Table 3.3: Simulation results of DGP2 (sparse endogeneous case)

			BLP (with cost IV)					BLP (without cost IV)					Shrinkage				
J	T		Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ
5	25	Bias	-0.12	-0.03	0.07	0.15	0.12	-0.08	-0.04	0.08	-0.08	0.11	-0.13	0.05	0.06	0.07	0.15
		SD	0.30	0.18	0.19	0.17	0.32	0.48	0.48	0.25	0.63	0.59	0.18	0.17	0.11	0.36	0.26
5	100	Bias	0.02	-0.05	-0.01	0.10	0.03	-0.29	0.37	0.06	0.09	0.32	-0.02	0.02	0.01	-0.04	0.03
		SD	0.16	0.10	0.11	0.14	0.18	0.65	1.03	0.16	0.79	0.96	0.06	0.06	0.03	0.06	0.08
15	25	Bias	0.04	-0.03	0.00	0.09	0.04	-0.13	0.41	-0.01	0.15	0.19	0.01	0.01	-0.01	-0.03	0.03
		SD	0.09	0.07	0.07	0.11	0.12	0.24	0.68	0.19	0.48	0.63	0.03	0.04	0.02	0.06	0.06
15	100	Bias	0.00	-0.02	0.03	0.03	0.02	0.02	-0.11	0.05	-0.00	0.13	0.01	0.00	-0.00	-0.04	0.02
		SD	0.09	0.10	0.05	0.04	0.11	0.35	0.80	0.07	0.01	0.63	0.02	0.01	0.01	0.02	0.04

(a) Comparison between strong BLP (with cost IV), weak BLP (without cost shock), and the Shrinkage method of [Lu and Shimizu, 2025].

			BLP (with cost IV)					BLP (without cost IV)					BLP (no cost/ competitors)				
J	T		Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ
5	25	Bias	-0.12	-0.03	0.07	0.15	0.12	-0.08	-0.04	0.08	-0.08	0.11	-0.36	0.06	0.28	0.00	0.47
		SD	0.30	0.18	0.19	0.17	0.32	0.48	0.48	0.25	0.63	0.59	1.88	1.93	0.78	0.00	2.14
5	100	Bias	0.02	-0.05	-0.01	0.10	0.03	-0.29	0.37	0.06	0.09	0.32	-0.41	1.11	0.01	0.00	0.57
		SD	0.16	0.10	0.11	0.14	0.18	0.65	1.03	0.16	0.79	0.96	1.02	2.25	0.25	0.00	1.91
15	25	Bias	0.04	-0.03	0.00	0.09	0.04	-0.13	0.41	-0.01	0.15	0.19	0.36	1.31	-0.39	0.00	0.94
		SD	0.09	0.07	0.07	0.11	0.12	0.24	0.68	0.19	0.48	0.63	0.98	5.05	1.10	0.00	3.25
15	100	Bias	0.00	-0.02	0.03	0.03	0.02	0.02	-0.11	0.05	-0.00	0.13	-0.19	0.96	0.05	0.00	0.47
		SD	0.09	0.10	0.05	0.04	0.11	0.35	0.80	0.07	0.01	0.63	0.82	2.17	0.16	0.00	1.74

(b) Comparison between Lu's BLP benchmarks and our proposed BLP variant without cost (competitor features).

Note: The bias/SD of ξ are the averages of (absolute value of) bias/SD of ξ_{jt}. Int = the intercept term $\bar{\xi}$.

Table 3.5: Simulation results of DGP3 (non-sparse exogeneous case)

			BLP (with cost IV)					BLP (without cost IV)					Shrinkage				
J	T		Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ
5	25	Bias	0.02	0.01	-0.01	0.09	0.02	0.10	-0.11	-0.02	0.04	0.10	0.09	-0.09	-0.05	0.09	0.28
		SD	0.18	0.10	0.11	0.13	0.36	0.26	0.28	0.09	0.12	0.46	0.24	0.11	0.16	0.19	0.40
5	100	Bias	-0.01	0.02	0.00	0.02	0.01	0.01	0.01	0.00	-0.11	0.07	0.04	-0.00	-0.03	-0.05	0.25
		SD	0.07	0.07	0.04	0.02	0.33	0.17	0.43	0.06	0.48	0.44	0.10	0.06	0.06	0.11	0.33
15	25	Bias	0.04	-0.06	0.01	0.03	0.03	-0.03	0.06	0.01	-0.02	0.06	-0.07	-0.00	0.05	-0.04	0.27
		SD	0.09	0.08	0.05	0.05	0.34	0.15	0.33	0.06	0.14	0.40	0.07	0.07	0.04	0.10	0.28
15	100	Bias	-0.01	-0.07	0.02	0.03	0.02	0.07	-0.18	0.03	-0.01	0.11	-0.02	-0.01	0.03	-0.06	0.26
		SD	0.05	0.10	0.03	0.04	0.34	0.31	0.51	0.05	0.39	0.58	0.09	0.03	0.06	0.04	0.29

(a) Comparison between strong BLP (with cost IV), weak BLP (without cost shock), and the Shrinkage method of [Lu and Shimizu, 2025].

			BLP (with cost IV)					BLP (without cost IV)					BLP (no cost/ competitors)				
J	T		Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ
5	25	Bias	0.02	0.01	-0.01	0.09	0.02	0.10	-0.11	-0.02	0.04	0.10	0.07	-0.17	0.10	0.00	0.20
		SD	0.18	0.10	0.11	0.13	0.36	0.26	0.28	0.09	0.12	0.46	0.49	1.17	0.34	0.00	0.97
5	100	Bias	-0.01	0.02	0.00	0.02	0.01	0.01	0.01	0.00	-0.11	0.07	3.32	-5.28	-0.51	0.00	3.64
		SD	0.07	0.07	0.04	0.02	0.33	0.17	0.43	0.06	0.48	0.44	7.31	11.92	1.16	0.00	9.86
15	25	Bias	0.04	-0.06	0.01	0.03	0.03	-0.03	0.06	0.01	-0.02	0.06	2.65	-8.78	1.06	0.00	5.74
		SD	0.09	0.08	0.05	0.05	0.34	0.15	0.33	0.06	0.14	0.40	7.59	28.75	3.54	0.00	17.99
15	100	Bias	-0.01	-0.07	0.02	0.03	0.02	0.07	-0.18	0.03	-0.01	0.11	0.77	-1.29	0.06	0.00	0.99
		SD	0.05	0.10	0.03	0.04	0.34	0.31	0.51	0.05	0.39	0.58	1.64	4.26	0.17	0.00	2.80

(b) Comparison between Lu's BLP benchmarks and our proposed BLP variant without cost (competitor features).

Note: The bias/SD of ξ are the averages of (absolute value of) bias/SD of ξ_{jt}. Int = the intercept term $\bar{\xi}$.

Table 3.7: Simulation results of DGP4 (non-sparse endogeneous case)

			BLP (with cost IV)					BLP (without cost IV)					Shrinkage				
J	T		Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ
5	25	Bias	0.09	-0.02	-0.08	0.11	0.11	0.15	-0.21	-0.09	0.19	0.23	-0.05	0.08	-0.00	-0.06	0.28
		SD	0.14	0.12	0.08	0.22	0.36	0.31	0.86	0.11	0.86	0.74	0.27	0.13	0.18	0.20	0.41
5	100	Bias	0.05	-0.00	-0.03	0.05	0.05	-0.07	0.16	-0.01	0.10	0.11	-0.01	0.06	-0.02	0.02	0.26
		SD	0.09	0.08	0.06	0.07	0.34	0.15	0.62	0.10	0.54	0.49	0.12	0.08	0.08	0.10	0.33
15	25	Bias	0.02	-0.03	0.01	0.08	0.03	0.08	-0.12	0.01	0.02	0.09	-0.09	0.04	0.04	-0.00	0.28
		SD	0.10	0.14	0.06	0.09	0.34	0.14	0.22	0.06	0.06	0.42	0.13	0.08	0.09	0.12	0.29
15	100	Bias	0.05	-0.03	0.00	0.03	0.05	0.09	-0.16	0.00	0.21	0.11	-0.02	0.06	-0.00	-0.03	0.27
		SD	0.05	0.13	0.02	0.05	0.34	0.20	0.42	0.05	0.49	0.50	0.09	0.03	0.06	0.05	0.28

(a) Comparison between strong BLP (with cost IV), weak BLP (without cost shock), and the Shrinkage method of [Lu and Shimizu, 2025].

			BLP (with cost IV)					BLP (without cost IV)					BLP (no cost/ competitors)				
J	T		Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ	Int	β_p	β_w	σ	ξ
5	25	Bias	0.09	-0.02	-0.08	0.11	0.11	0.15	-0.21	-0.09	0.19	0.23	0.11	0.22	-0.08	0.00	0.14
		SD	0.14	0.12	0.08	0.22	0.36	0.31	0.86	0.11	0.86	0.74	0.19	0.34	0.10	0.00	0.44
5	100	Bias	0.05	-0.00	-0.03	0.05	0.05	-0.07	0.16	-0.01	0.10	0.11	-0.04	0.32	-0.02	0.00	0.18
		SD	0.09	0.08	0.06	0.07	0.34	0.15	0.62	0.10	0.54	0.49	0.41	0.91	0.07	0.00	0.80
15	25	Bias	0.02	-0.03	0.01	0.08	0.03	0.08	-0.12	0.01	0.02	0.09	0.30	0.01	0.00	0.00	0.32
		SD	0.10	0.14	0.06	0.09	0.34	0.14	0.22	0.06	0.06	0.42	0.52	0.93	0.09	0.00	0.86
15	100	Bias	0.05	-0.03	0.00	0.03	0.05	0.09	-0.16	0.00	0.21	0.11	-0.14	0.87	0.05	0.00	0.36
		SD	0.05	0.13	0.02	0.05	0.34	0.20	0.42	0.05	0.49	0.50	0.90	1.76	0.14	0.00	1.55

(b) Comparison between Lu's BLP benchmarks and our proposed BLP variant without cost (competitor features).

Note: The bias/SD of ξ are the averages of (absolute value of) bias/SD of ξ_{jt}. Int = the intercept term $\bar{\xi}$.

c Benchmark models used in Section 4 of Lu (2025)

The Bayesian Shrinkage method

This is the proposed method in [Lu and Shimizu, 2025] based on BLP random-coefficients logit framework for differentiated products with market–product unobservables. In market t with products $j = 0, 1, \ldots, J_t$ (where $j = 0$ is the outside option), the indirect utility is

$$u_{ijt} = \mathbf{x}_{jt}^{\top}\boldsymbol{\beta}_i + \xi_{jt} + \varepsilon_{ijt}, \tag{3.1}$$

where $\mathbf{x}_{jt}$ are observed characteristics (including an endogenous component such as price), $\boldsymbol{\beta}_i$ are random coefficients, ξ_{jt} is the market–product demand shock (unobserved to the econometrician but observed by firms), and ε_{ijt} is i.i.d. type-I extreme value. The implied market share is the standard random-coefficients logit share system. propose identifying and estimating the model without IVs by assuming a sparsity structure in the market–product unobservables ξ_{jt}. Lu and Shimizu [2025] decompose the shock as

$$\xi_{jt} = \bar{\xi}_t + \eta_{jt}, \tag{3.2}$$

and impose a spike-and-slab (shrinkage) prior on η_{jt} so that many η_{jt}'s are shrunk toward 0 (products share the same market-level component $\bar{\xi}_t$), while some remain free to deviate. Under the paper's sparsity conditions, this structure can restore identification and support likelihood-based Bayesian inference without relying on the IV restriction.

BLP Estimator.

The standard BLP approach proceeds Berry et al. [1995] in two conceptual steps: (i) invert demand to recover ξ_{jt} as a function of observed shares and candidate preference parameters, and (ii) use IV moment conditions to estimate the preference parameters by the generalized method of moments (GMM). The original BLP demand model can be expressed as:

$$u_{ijt} = \mathbf{x}_{jt}^{\top}\boldsymbol{\beta}_i + \xi_{jt} + \varepsilon_{ijt},$$

where $\mathbf{x}_{jt} = [\tilde{\mathbf{x}}_{jt}, p_{jt}]$ with $\tilde{\mathbf{x}}_{jt}$ denoting exogenous variables and p_{jt} denoting the endogenous price variable. Let $\boldsymbol{\beta}_i = \bar{\boldsymbol{\beta}} + \mathrm{Diag}(\boldsymbol{\sigma})\mathbf{v}_i$ where $\bar{\boldsymbol{\beta}} = [\bar{\tilde{\boldsymbol{\beta}}}, \bar{\beta}_{\text{price}}]$ and $\mathbf{v}_i \sim P_{\mathbf{v}} = \mathcal{N}(\mathbf{0}, \mathbf{I}_{d_X})$. Then, we can rewrite the utility as

$$u_{ijt} = \delta_{jt} + \mu_{ijt} + \varepsilon_{ijt},$$

where $\delta_{jt} = \mathbf{x}_{jt}^{\top}\bar{\boldsymbol{\beta}} + \xi_{jt}$ is the mean utility and $\mu_{ijt} = \mathbf{x}_{jt}^{\top}\mathrm{Diag}(\boldsymbol{\sigma})\mathbf{v}_i$ captures the heterogeneity. For each market t and product j, the model implies a market share mapping

$\sigma_{jt}(\boldsymbol{\delta}_t, \boldsymbol{\sigma})$, where $\boldsymbol{\delta}_t = [\delta_{jt}]$ is the mean utility vector for market t. The market share of product j in market t then yields as:

$$\sigma_{jt}(\boldsymbol{\delta}_t, \boldsymbol{\sigma}) = \int \frac{\exp(\delta_{jt} + \mu_{ijt})}{1 + \sum_{k=1}^{J_t} \exp(\delta_{kt} + \mu_{ikt})} dP_{\mathbf{v}}(\mathbf{v}_i).$$

Given observed shares s_{jt}, BLP recovers $\boldsymbol{\delta}_t$ by inverting the system $s_{jt} = \sigma_{jt}(\boldsymbol{\delta}_t, \boldsymbol{\sigma})$ using the contraction mapping

$$\boldsymbol{\delta}_{jt}^{k+1} \;=\; \boldsymbol{\delta}_{jt}^{k} + \log s_{jt} - \log \sigma_{jt}(\boldsymbol{\delta}_t, \boldsymbol{\sigma}),$$

iterated until convergence to a fixed point $\delta_{jt}^*(\boldsymbol{\sigma})$.

BLP then uses instrumental variables such as:

$$\mathbf{z}_{jt} = [\mathbf{w}_{jt},\ \text{sum of } \tilde{\mathbf{x}}_{kt} \text{ for } k \neq j,\ \text{sum of rival firm characteristics}],$$

where $\mathbf{w}_{jt}$ is a cost shifter.

Given δ_{jt}, the unobserved demand shock is obtained as the structural error

$$\xi_{jt}(\boldsymbol{\theta}) = \delta_{jt}^*(\boldsymbol{\sigma}) - \mathbf{x}_{jt}^\top \bar{\boldsymbol{\beta}},$$

where $\boldsymbol{\theta} = (\boldsymbol{\sigma}, \bar{\boldsymbol{\beta}})$. Now, we use GMM to estimate $\boldsymbol{\theta}$ by imposing orthogonality conditions with instruments $\mathbf{z}_{jt}$, typically through $\mathbb{E}[\mathbf{z}_{jt}\xi_{jt}(\boldsymbol{\sigma})] = \mathbf{0}$, and minimizing the quadratic criterion. For this purpose, a 2-stage optimization is employed to find $\boldsymbol{\theta}$.

In the first optimization stage, we minimize the following quadratic objective function:

$$\widehat{\boldsymbol{\theta}}_1 \;\in\; \underset{\boldsymbol{\theta}}{\mathsf{argmin}}\ \mathbf{g}(\boldsymbol{\theta})^\top \mathbf{S}_1^{-1} \mathbf{g}(\boldsymbol{\theta}), \qquad \mathbf{g}(\boldsymbol{\theta}) = \frac{1}{TJ} \sum_{t=1}^{T} \sum_{j=1}^{J} \mathbf{z}_{jt} \xi_{jt}(\boldsymbol{\theta}),$$

where $\mathbf{S}_1 = \frac{1}{TJ} \sum_{t,j} \mathbf{z}_{jt} \mathbf{z}_{jt}^\top$. Using $\widehat{\boldsymbol{\theta}}_1$, we then solve a more sophisticated optimization problem in the second stage formulated as

$$\widehat{\boldsymbol{\theta}}_2 \;\in\; \underset{\boldsymbol{\theta}}{\mathsf{argmin}}\ \mathbf{g}(\boldsymbol{\theta})^\top \widehat{\mathbf{S}}_2^{-1} \mathbf{g}(\boldsymbol{\theta}), \qquad \widehat{\mathbf{S}}_2 = \frac{1}{T} \sum_{t=1}^{T} \widehat{\mathbf{g}}_t(\widehat{\boldsymbol{\theta}}_1) \widehat{\mathbf{g}}_t(\widehat{\boldsymbol{\theta}}_1)^\top,$$

where $\widehat{\mathbf{g}}_t(\widehat{\boldsymbol{\theta}}_1) = \frac{1}{J} \sum_{j=1}^{J} \mathbf{z}_{jt} \xi_{jt}(\widehat{\boldsymbol{\theta}}_1)$.

In each stage, we perform alternating optimization over $\bar{\boldsymbol{\beta}}$ and $\boldsymbol{\sigma}$. Since $\xi_{jt}(\boldsymbol{\theta})$ is a linear function of $\bar{\boldsymbol{\beta}}$, the GMM optimization problem in each stage, given $\boldsymbol{\sigma}$, reduces to a simple least-squares problem (hence the name two-stage least squares, or 2SLS), for which a closed-form solution exists.

Benchmark 1: BLP "with cost IV" (strong IVs)

In the BLP benchmark algorithm used in [Lu and Shimizu, 2025], price (or the endogenous "price-like" attribute) is generated from a reduced-form pricing equation with an additive cost shock u_{jt}, and the first benchmark uses an IV $\mathbf{z}_{jt}$ that includes functions of u_{jt} in addition to functions of exogenous characteristics w_{jt} as

$$\mathbf{z}_{jt} = \left(1, w_{jt}, w_{jt}^2, u_{jt}, u_{jt}^2\right).$$

This benchmark is labeled "BLP with cost IV" in the simulations in Section 4 of [Lu and Shimizu, 2025] and emphasize that u_{jt} is typically unobserved in real data, so this is a "best-case" strong-IV benchmark under the assumed DGP.

Benchmark 2: BLP "without cost IV" (weak IVs)

This benchmark utilizes a similar BLP estimator, but the instrument set omits the cost shock and uses only functions of the observed exogenous characteristic(s), e.g.,

$$\mathbf{z}_{jt} = \left(1, w_{jt}, w_{jt}^2, w_{jt}^3, w_{jt}^4\right).$$

Lu and Shimizu [2025] use this design to mimic a common applied setting: researchers observe demand shifters but do not observe clean cost shifters, leading to weak or invalid instruments and poor finite-sample behavior.

c.1 Assumptions needed for BLP+IV with price endogeneity

BLP-style endogeneity arises because firms set an endogenous attribute (classically price) using information in ξ_{jt} that econometricians do not observe, implying

$$\mathrm{Cov}(p_{jt}, \xi_{jt}) \neq 0 \tag{3.3}$$

(and similarly for any endogenous element of $\mathbf{x}_{jt}$).

A minimal set of assumptions for BLP+IV to identify and consistently estimate demand parameters in the presence of price endogeneity includes:

A1. Demand inversion / uniqueness. For a given parameterization of heterogeneity, the mapping from mean utilities δ_{tj} to shares should be invertible so that ξ_{jt} can be recovered from observed shares and candidate parameters.

A2. Instrument exogeneity (exclusion/validity). Instruments must be orthogonal to the unobserved demand shock:

$$\mathbb{E}\left[\xi_{jt} \mid \mathbf{z}_{jt}\right] = 0 \quad \text{(or equivalently } \mathbb{E}[\xi_{jt}\mathbf{z}_{jt}] = 0 \text{ under appropriate conditions).}$$

This is the core identifying restriction used in BLP GMM. Hence, when choosing $\mathbf{z}_{jt} = \mathbf{1}$ as an IV, we need to ensure the unobserved shocks ξ_{jt} are centralized.

A3. Instrument relevance (rank/strength). $\mathbf{z}_{jt}$ must shift the endogenous variable(s) in $\mathbf{x}_{jt}$ sufficiently so that the GMM moments are informative (informally: a strong first stage; formally: rank conditions).

A4. Correct specification of the structural demand system. The inversion-based residual $\sigma_{jt}^{-1}(\mathbf{s}_t, \boldsymbol{\sigma})$ must be the accurate structural unobservable. Misspecification can invalidate moment conditions even when $\mathbf{z}_{jt}$ is exogenous.

c.2 Observed vs unobserved variables justifying IV

The IV approach is justified when some variables affecting both the endogenous attribute and utility are observed by the firm/bank/merchant but not by the econometrician, which is precisely the role of ξ_{jt} in the BLP model in [Lu and Shimizu, 2025].

Observed variables In typical merchant-offer datasets one may observe:

- Offer terms: required spend threshold x, rebate/discount y, percentage off, cap, expiry, eligibility rules.
- Merchant/category identifiers and timing (campaign start/end).
- Redemption outcomes: activation/redeem indicator, or aggregate redemption shares.
- Some consumer observables (e.g., spending history, demographics) if available (the Lu and Shimizu [2025] likelihood can be extended when individual covariates exist, but their baseline discusses aggregate data).

Unobservable variables Examples of unobservables that may affect both offer "price" and redemption demand (analogous to the quality shocks ξ_{jt}) that may create endogeneity include:

- Unobserved merchant attractiveness or latent demand shocks (e.g., local events, brand perception, unmeasured quality).

- Unobserved marketing intensity/placement (e.g., in-app visibility, email targeting intensity) that shifts redemption probabilities.
- Private targeting signals—such as the issuer's predicted incremental lift or latent preferences—used to set y or x and to determine offer eligibility.

If such models, ξ_{jt} enters utility and also affects the offer terms (or the probability of being targeted), then naive estimation of the effect of y (or the effective net price) is biased without IV or an alternative identification strategy.

c.3 How to find suitable instruments

In BLP framework, one should seek variables that:

1. **Shift the endogenous offer term** (they are relevant), and
2. **Do not directly enter consumer utility except through the endogenous term** after conditioning on controls and fixed effects (exogeneity/exclusion).

In applied work, this usually means cost shifters, policy/rule shocks, or competitor-driven supply shifters that affect the bank/merchant's offer design but are plausibly orthogonal to the unobserved demand shock.

In merchant offers, APR and annual fee are typically card-level attributes rather than offer-level attributes; the more natural endogenous "price-like" variables are the offer generosity and constraints, e.g. y (rebate), x (threshold), discount rate, caps, and expiry. Thus, IVs should generally be constructed to shift these offer terms rather than APR/annual fee unless the model is explicitly a joint model of card choice and offer redemption.

c.4 Example IVs for credit-card offers

Below are three cases of instrumental variables that are often defensible in the merchant-offer context, stated in terms of the required IV assumptions (exogeneity and relevance as above). These are, in fact, merchant-offer analogs of "cost shifters" and "supply shifters" that BLP uses to create $\mathbf{z}_{jt}$ satisfying $\mathbb{E}[\xi_{jt}\mathbf{z}_{jt}] = 0$ while moving the endogenous regressor.

IV1: Merchant cost shifters interacted with category exposure

Example. Use merchant input-cost factors (e.g., commodity price indices for fuel/groceries; local wage/rent indices for services) interacted with merchant category and timing to instrument the rebate y or discount rate.

Relevance. Higher marginal costs reduce a merchant's willingness to fund deep discounts, shifting y and related terms.

Exogeneity argument. Conditional on rich fixed effects (merchant, time, category×time), these cost shifters affect utility primarily through the offer terms, not through unobserved idiosyncratic demand shocks for a particular targeted offer.

IV2: Network/interchange or settlement-fee schedule shocks

Example. Instruments constructed from changes in interchange/settlement costs (or network rule changes) interacted with merchant category exposure.

Relevance. Such changes shift the economics of card-funded or merchant-funded promotions, moving offer generosity/caps.

Exogeneity argument. These are plausibly predetermined at the offer level and (after time effects) orthogonal to the within-offer unobserved demand shock ξ_{jt}; they shift supply-side incentives rather than latent tastes.

IV3: Competitor offer-pressure measures (leave-one-out)

Example. For each merchant-category-week, use rivals' promotion intensity (e.g., number/average generosity of competing offers in the same category and geography), excluding the own merchant.

Relevance. Competitive pressure induces matching or strategic changes in y, x, caps, and expiry.

Exogeneity argument. With category×time fixed effects and local market controls held constant, rivals' promotion choices are likely less correlated with the focal offer's unobserved demand shock than the focal offer's own choices are. This makes them more plausible as instruments.

c.5 Other benchmark models

As discussed in the previous section, we proposed another BLP variant as a benchmark that relies on competitor characteristics. In this case, the instrumental variables are defined as:

$$\mathbf{z}_{jt} = [1, w_{jt}, \sum_{k \neq j} w_{kt}, \sum_{k \neq j} (w_{jt} - w_{kt})^2]$$

As shown in the simulation results, this BLP variant accurately estimates the variance of the price taste parameter (σ_0). However, its performance in estimating the other parameters is comparatively poor.

Another approach to expand the simulation benchmarks is to add two estimators that impose alternative identifying structure relative to (i) BLP-GMM with IVs and (ii) Lu's

sparsity-based Bayesian shrinkage. First, one can benchmark against an interactive fixed effects / factor-structured shock specification like the one proposed in Bai [2009], with

$$\xi_{jt} = \boldsymbol{\lambda}_j^\top \mathbf{f}_t + \varepsilon_{jt}, \qquad \boldsymbol{\lambda}_j \in \mathbb{R}^L,\ \mathbf{f}_t \in \mathbb{R}^L,\ L \ll \min(J, T),$$

which is an intuitive competing restriction: instead of assuming many shocks are exactly (or nearly) zero/equal within a market (sparsity), we assume market–product shocks are driven by a small number of latent market-level forces with heterogeneous product exposures (low rank). Logically, this targets the same core difficulty emphasized by Lu and Shimizu [2025]. Without such low-rank structure, the high-dimensional vector $\{\xi_{jt}\}$ is too flexible, and one must either rely on strong IV moment conditions (BLP) or impose restrictions on ξ_{jt} to restore identification and stable inference; a rank-L factor model reduces the effective number of free shock parameters from JT to approximately $JL + TL$ (up to rotations), providing a clean "structured-ξ_{jt}" comparison to sparsity-based identification.

Second, a natural additional benchmark is a control-function approach for the endogenous offer term (e.g., y_{jt} or an effective discount), which differs from BLP-IV but addresses the same endogeneity problem. Concretely:

1. Estimate a first-stage model for the endogenous offer term using $\mathbf{z}_{jt}$ (and controls) and recover a residual $\widehat{\nu}_{jt}$.

2. Include $\widehat{\nu}_{jt}$ (or a flexible function of it) as an additional regressor in the utility index to control for correlation between the endogenous term and ξ_{jt}.

The mathematical logic is that if the endogeneity arises through a reduced-form relationship $y_{jt} = g(\mathbf{z}_{jt}, \mathbf{x}_{jt}) + \nu_{jt}$ with ν_{jt} capturing the component correlated with ξ_{jt}, then conditioning on (a function of) ν_{jt} can render the remaining variation in y_{jt} orthogonal to ξ_{jt}, allowing consistent estimation in the second stage using a different restriction than pure IV-GMM moments. This therefore provides a robust and informative benchmark because it targets the same "endogenous price/offer" mechanism discussed by Lu and Shimizu [2025] while changing how identification is achieved: it remains close in spirit to the BLP endogeneity setup, but shifts the burden from instrument strength in moment conditions to a residual-inclusion (control-function) correction. [Petrin and Train, 2010].

d Context-Dependent Choice Model with Sparse Shocks

This section introduces a hybrid discrete choice model that integrates the sparse market–product shock framework of Lu and Shimizu [2025] with the deep context-dependent choice architecture of Zhang et al. [2025]. The goal is to preserve the economic structure and interpretability of random-coefficients logit models with additive demand shocks,

while allowing for flexible, context-driven substitution patterns that relax the Independence of Irrelevant Alternatives (IIA) property in a controlled, data-driven manner.

The two building blocks address complementary limitations of standard discrete choice models. Lu and Shimizu [2025] target endogeneity and high-dimensional unobserved heterogeneity by imposing a sparsity structure on market–product demand shocks and estimating these shocks in a Bayesian framework. In contrast, Zhang et al. [2025] propose a neural architecture that maps the entire observed choice set into context-dependent utilities, typically trained by minimizing a cross-entropy objective in a deterministic setting. Our contribution is to embed Lu-style sparse shocks as additive intercepts within a Zhang-style context-dependent utility specification, yielding a model in which context effects originate from observables, while unobserved quality remains an additive component.

Implementation overview. Operationally, the proposed approach combines two components:

1. a DeepHalo encoder that maps item features and availability into a context-dependent representation (latent features or logits), and

2. a Bayesian econometric layer that imposes shrinkage on market–product shocks and is estimated via MCMC (Metropolis–Hastings and Gibbs updates).

The DeepHalo component follows the `choicelearn` design pattern, in which the model exposes `trainable_weights` and provides differentiable computations of utilities for gradient-based training [Auriau et al., 2024]. The accompanying codebase also includes (i) a teacher data-generating process (DGP) with a neural encoder to induce context dependence, (ii) two neural baselines, and (iii) a hybrid sparse estimator implemented as an MCEM procedure with an MCMC-based E-step, together with a simulation test suite for predictive evaluation and sparsity recovery.

d.1 Setup and notation

Markets are indexed by $t = 1, \ldots, T$, individuals by $i = 1, \ldots, N_t$, and products by $j \in \{1, \ldots, J\}$. Individual i in market t faces a choice set

$$S_{it} \subseteq \{1, \ldots, J\},$$

and each product j in market t is characterized by observable attributes $\mathbf{x}_{jt}$ (e.g., price, brand, size). Following Lu and Shimizu [2025], we introduce a market–product shock ξ_{jt} and decompose it as

$$\xi_{jt} = \bar{\xi}_t + \eta_{jt}, \tag{3.4}$$

where $\bar{\xi}_t$ captures market-level unobserved demand shocks and η_{jt} represents product-specific deviations. A sparsity prior is imposed on η_{jt} so that, within each market, only a subset of products exhibits non-negligible deviations from the market intercept.

Let $f_{\boldsymbol{\theta}}$ denote a permutation-equivariant neural network (in the spirit of Zhang et al. [2025]) that maps the set of observable item features in the choice set into context-dependent embeddings. For each market t, define the matrix of observable features as

$$\mathbf{X}_{S_t} = [\mathbf{x}_{jt}]_{j \in S_t} \in \mathbb{R}^{d_X \times |S_t|}.$$

The encoder produces a context-aware embedding for each product:

$$\mathbf{z}_{jt} = [f_{\boldsymbol{\theta}}(\mathbf{X}_{S_t})]_j \in \mathbb{R}^{d_Z}, \quad \forall j \in S_t \tag{3.5}$$

By construction, $\mathbf{z}_{jt}$ depends on the entire observable context $\mathbf{X}_{S_t}$, but not on the unobserved shock ξ_{jt}. This separation ensures that context effects arise from observables, while unobserved quality remains additive, consistent with the econometric structure in Lu and Shimizu [2025].

Utility and market share specification

Let $\boldsymbol{\beta}_i \in \mathbb{R}^d$ denote individual i's taste vector over the d-dimensional context-aware features $\mathbf{z}_{jt}$. The utility of individual i for product j in market t is

$$u_{ijt} = \mathbf{z}_{jt}^\top \boldsymbol{\beta}_i + \xi_{jt} + \varepsilon_{ijt}, \tag{3.6}$$

where ε_{ijt} are i.i.d. type-I extreme value errors. This specification has three key components: (i) $\mathbf{z}_{jt}^\top \boldsymbol{\beta}_i$ captures context-dependent tastes with heterogeneous valuations, (ii) $\xi_{jt} = \bar{\xi}_t + \eta_{jt}$ enters as an additive intercept independent of the observed context, and (iii) ε_{ijt} provides the familiar logit error structure. Let $\boldsymbol{\beta}_t = \bar{\boldsymbol{\beta}} + \mathrm{Diag}(\exp(\mathbf{r}))\mathbf{v}_i$, where $\mathbf{v}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$. Then given $(\boldsymbol{\theta}, \bar{\boldsymbol{\beta}}, \mathbf{r}, \boldsymbol{\xi})$, the market share for product $j \in S_t$ yields as:

$$\sigma_{jt}(\boldsymbol{\theta}, \bar{\boldsymbol{\beta}}, \mathbf{r}, \boldsymbol{\xi}) = \int \frac{\exp(\mathbf{z}_{jt}(\boldsymbol{\theta})^\top \boldsymbol{\beta}_i + \xi_{jt})}{1 + \sum_{k \in S_{it}} \exp(\mathbf{z}_{kt}(\boldsymbol{\theta})^\top \boldsymbol{\beta}_i + \xi_{kt})} dP_{\mathbf{v}}(\mathbf{v}_i).$$

d.2 Simulation study design

Data-generating process (DGP)

We simulate markets $t = 1, \ldots, T$, each with a choice set S_t of fixed size J drawn from a universe of M items. Each item has features $\mathbf{x}_{jt} \in \mathbb{R}^{d_X}$. The data generation process is as follows:

1. Generate sets and features: Draw item features $\mathbf{x}_{jt} \sim \mathcal{N}(0, I_d)$ and, for each market,

sample a subset S_t of size J.

2. Generate "true" DeepHalo utilities: Instantiate a teacher DeepHalo network with fixed parameters $\boldsymbol{\theta}^\star$ and compute

$$\mathbf{z}^\star_{jt} = [f_{\boldsymbol{\theta}^\star}(\mathbf{X}_{S_t})]_j \in \mathbb{R}^{d_Z}, \quad \forall j \in S_t \tag{3.7}$$

The encoder maps d_X-dimensional inputs into d_Z-dimensional context features (controlled by `out_dim` in the code).

3. Generate random heterogeneous tattie parameters by $\boldsymbol{\beta}^\star_i = \bar{\boldsymbol{\beta}}^\star + \mathrm{Diag}(\exp(\mathbf{r}^\star))\mathbf{v}_i$, where

$$\begin{aligned}\bar{\boldsymbol{\beta}}^\star &\sim \mathcal{N}(\mathbf{0}, \sigma^2_\beta \mathbf{I}_{d_Z}) \\ \mathbf{r}^\star &\sim \mathcal{N}(\mathbf{0}, \sigma^2_r \mathbf{I}_{d_Z}) \\ \mathbf{v}_i &\sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{d_Z})\end{aligned}$$

4. Add sparse market–product shocks: Generate market intercepts $\bar{\xi}^\star_t \sim \mathcal{N}(0, \sigma^2_\xi)$ and sparse deviations

$$\eta^\star_{jt} = \begin{cases} 0, & \text{w.p. } \pi, \\ \mathcal{N}(0, \sigma^2_\eta), & \text{w.p. } 1-\pi, \end{cases} \tag{3.8}$$

independently across (j,t), with $\pi \in \{0.8, 0.9, 0.95\}$ in our experiments. Set $\xi^\star_{jt} = \bar{\xi}^\star_t + \eta^\star_{jt}$.

5. Simulate choices: For each market t, generate N_t i.i.d. choices using the market shares as a probability mass function (over products)

$$\begin{aligned}P^\star(j \mid S_t) &= \sigma_{jt}(\boldsymbol{\theta}^\star, \bar{\boldsymbol{\beta}}^\star, \mathbf{r}^\star, \boldsymbol{\xi}^\star) \\ &= \int \frac{\exp(\mathbf{z}^\star_{jt}(\boldsymbol{\theta}^\star)^\top(\bar{\boldsymbol{\beta}}^\star + \mathrm{Diag}(\mathbf{r}^\star)\mathbf{v}) + \xi_{jt})}{1 + \sum_{k \in S_{it}} \exp(\mathbf{z}^\star_{kt}(\boldsymbol{\theta}^\star)^\top(\bar{\boldsymbol{\beta}}^\star + \mathrm{Diag}(\mathbf{r}^\star)\mathbf{v}) + \xi_{kt})} dP_{\mathbf{v}}(\mathbf{v}).\end{aligned}$$

Benchmarks

We compare three estimators (the proposed + two benchmarks) fitted on the simulated training markets:

1. **DeepHalo baseline:** encoder + $(\bar{\boldsymbol{\beta}}, \mathbf{r})$, with $\bar{\xi}_t = \eta_{jt} = 0$.
2. **DeepHalo + market intercepts only:** encoder + $(\bar{\boldsymbol{\beta}}, \mathbf{r}, \bar{\xi}_t)$, with $\eta_{jt} = 0$.
3. **Proposed hybrid sparse model:** encoder trained within MCEM, with $(\bar{\boldsymbol{\beta}}, \mathbf{r}, \bar{\xi}_t, \boldsymbol{\eta})$ inferred by MCMC in the E-step under a sparsity prior.

In all methods, the underlying data are aggregate counts q_{tj} (including the outside option at index 0), and likelihood evaluation uses the simulated-share mixed-logit kernel implemented in `compute_probs_and_ll_batch_masked`.

Given latent item representations $\mathbf{z}$ produced by the encoder and simulated shares σ_{tj}, the aggregate log-likelihood is

$$\ell = \sum_{t=1}^{T} \sum_{j=0}^{M} q_{tj} \log(\sigma_{tj}), \qquad \text{NLL} = -\frac{\ell}{\sum_t \sum_j q_{tj}}.$$

This normalization is used both in SGD training losses and in the test metric `compute_prediction_nll`.

Baseline 1: DeepHalo (no $\bar{\xi}_t$, no η_{jt})

The baseline estimator trains a `DeeHaloEncoder` jointly with mixed-logit taste parameters $(\bar{\boldsymbol{\beta}}, r)$, imposing $\bar{\xi}_t = 0$ and $\eta_{jt} = 0$. The SGD objective is

$$\mathcal{L}_{\text{baseline}} = \text{NLL}(\mathbf{q} \mid \mathbf{Z}, \bar{\boldsymbol{\beta}}, \mathbf{r}, \bar{\xi}_t = 0, \eta_{jt} = 0) + \lambda_{\text{net}} \sum_{\mathbf{w} \in \mathcal{W}_{\text{enc}}} \|\mathbf{w}\|_2^2,$$

where λ_{net} corresponds to `nnl2`. At evaluation time, this method is scored by strict NLL (no test-market adaptation) and by adapted NLL, where only $\bar{\xi}_t$ is optimized on the test markets.

Baseline 2: DeepHalo + $\bar{\xi}_t$ only

The $\bar{\xi}_t$-only estimator augments the baseline with per-market intercepts $\bar{\xi}_t$ while maintaining $\eta_{tj} = 0$. Its SGD objective adds a quadratic penalty on $\bar{\xi}_t$:

$$\mathcal{L}_{\bar{\xi}_t\text{-only}} = \text{NLL}(\mathbf{q} \mid \mathbf{Z}, \bar{\boldsymbol{\beta}}, \mathbf{r}, \bar{\xi}_t, \eta_{tj} = 0) + \lambda_{\text{net}} \sum_{\mathbf{w} \in \mathcal{W}_{\text{enc}}} \|\mathbf{w}\|_2^2 + \lambda_{\bar{\xi}_t} \sum_{t=1}^{T} \bar{\xi}_t^2.$$

Evaluation again reports strict NLL and adapted NLL, where $\bar{\xi}_t$ is inferred on test markets via optimization.

Proposed hybrid method: DeepHalo + sparse MCEM

The proposed estimator couples the DeepHalo neural context encoder with a Bayesian sparse random-coefficients logit layer via Monte Carlo EM (MCEM) comprised of expectation and maximization steps (E-step and M-step).

E-Step: In this step, conditional on current encoder parameters and outputs $\mathbf{Z}(\boldsymbol{\theta}^k)$, MCMC samples the posterior of random effects:

$$p(\bar{\xi}_t, \eta_{tj}, \gamma_{tj}, \phi_t \mid \mathbf{q}, \mathbf{Z}(\boldsymbol{\theta}^k), \bar{\boldsymbol{\beta}}^{(k)}, \mathbf{r}^{(k)})$$

where $\bar{\boldsymbol{\beta}}^{(k)}$, $\mathbf{r}^{(k)}$ are fixed at current SGD values (MCMC step sizes set to 10^{-12}). This step generates sample draws of the random parameters and the posterior means $\mathbb{E}[\bar{\xi}_t^{(k)}]$, $\mathbb{E}[\eta_{tj}^{(k)}]$.

M-Step: This step uses Adam SGD optimizer over encoder weights + structural parameters $\bar{\boldsymbol{\beta}}$, $\mathbf{r}$ to minimize the MCMC-approximated expected complete-data log-likelihood. Hence, the loss function for the M-step yields as:

$$\mathcal{L}_{\text{MCEM-M}}(\boldsymbol{\theta}, \bar{\boldsymbol{\beta}}, \mathbf{r}) = \text{NLL}\big(\mathbf{q} \mid \mathbf{z}_t(\boldsymbol{\theta}), \bar{\boldsymbol{\beta}}, \mathbf{r}, \mathbb{E}[\bar{\xi}_t^{(k)}], \mathbb{E}[\eta_{tj}^{(k)}]\big) + \lambda_{\text{net}} \sum_{\mathbf{w} \in \mathcal{W}_{\text{enc}}} \|\mathbf{w}\|_2^2$$

The separation of $\bar{\boldsymbol{\beta}}$ and $\mathbf{r}$ from the remaining MCMC parameters is motivated by their tight coupling with the encoder weights $\boldsymbol{\theta}$, as they appear as multiplicative factors. Because $\boldsymbol{\theta}$ must be trained via SGD, keeping $\bar{\boldsymbol{\beta}}$ and $\mathbf{r}$ in the SGD step allows the optimizer to jointly coordinate the latent embedding space and the utility coefficients. If $\bar{\boldsymbol{\beta}}$ and $\mathbf{r}$ were updated in the MCMC step, the neural network would be trying to optimize against a moving target of posterior distributions, making the representation learning highly unstable. At convergence, the encoder produces market representations $\mathbf{Z}^\star(\boldsymbol{\theta}^\star)$ jointly optimized with SGD-calibrated parameters $\{\bar{\boldsymbol{\beta}}^\star, \mathbf{r}^\star\}$. Test prediction adapts only $\bar{\xi}_t$ (setting $\eta_{tj} = 0$ on test markets), so improvements reflect better representation learning under MCMC-updated taste parameters rather than direct test-set η_{tj} usage.

d.3 Numerical analysis

Table 3.9 summarizes the mean and standard deviation of key metrics across Monte Carlo replications. We report: (i) strict test (test) NLL, (ii) adapted test NLL (with test-market intercept $\bar{\xi}_t$ optimized), and (iv) parameter ($\bar{\boldsymbol{\beta}}$, $\mathbf{r}$) and encoder ($\boldsymbol{\theta}$) distances to the teacher DGP. As shown in this table, under the adapted evaluation protocol, the proposed hybrid estimator achieves the lowest test NLL on average (2.577 versus 2.628 for the $\bar{\xi}_t$-only baseline and 2.655 for the baseline DeepHalo).

As illustrated in Figure 3.6, the NLL loss on training samples consistently decreases over the MCEM iterations of the proposed hybrid sparse approach. Figure 3.5 further compares the performance of our proposed approach against the benchmark on test data as a function of the sparsity parameter $\pi = \mathbb{P}(\eta_{tj} = 0)$. This figure indicates that the hybrid sparse method consistently outperforms the baselines in terms of adapted test NLL across the considered values of π. The $\bar{\xi}_t$-only baseline uniformly improves upon the model that omits market intercepts, and the proposed hybrid sparse MCEM method outperforms both. As π increases (i.e., shocks become sparser), test NLL decreases for all methods, reflecting a simpler latent shock environment. These patterns are broadly consistent with the DGP: because the data include market-level heterogeneity and sparse product-level deviations, allowing $\bar{\xi}_t$ and (during training) η_{jt} improves predictive fit when

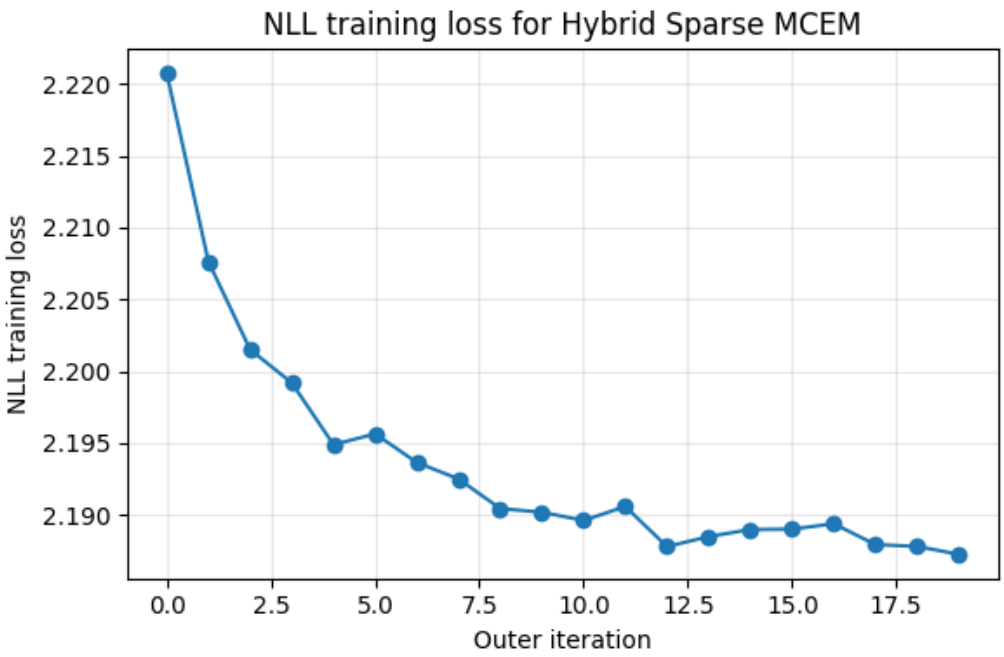


Figure 3.5: Training NLL loss versus outer-loop iterations of the proposed MCEM method for hybrid sparse Bayesian modeling of DeepHalo choice model with sparse unobserved market-product shocks.

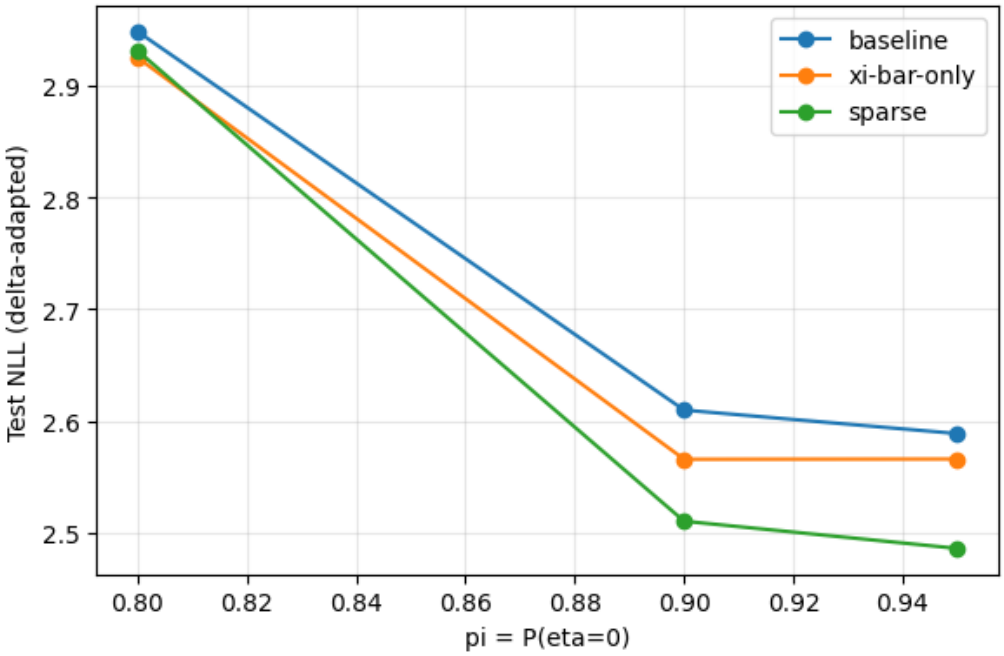


Figure 3.6: Test NLL loss under the $\bar{\xi}_t$-adapted evaluation protocol (lower is better) as a function of sparsity $\pi = \mathbb{P}(\eta_{tj} = 0)$.

market-level calibration is permitted at test time.

Metric	Baseline DeepHalo	DeepHalo + $\bar{\xi}_t$ only	DeepHalo + $\bar{\xi}_t$+ sparse η_{jt}
Strict test NLL (↓)	2.675 (0.332)	2.655 (0.367)	2.598 (0.335)
Adapted test NLL (↓)	2.655 (0.351)	2.628 (0.345)	2.577 (0.327)
$\|\hat{\bar{\boldsymbol{\beta}}} - \bar{\boldsymbol{\beta}}^\star\|_2$ (↓)	1.117 (0.534)	1.081 (0.531)	1.063 (0.571)
$\|\hat{\mathbf{r}} - \mathbf{r}^\star\|_2$ (↓)	0.535 (0.288)	0.508 (0.283)	0.487 (0.269)
$\|\hat{\mathbf{Z}} - \mathbf{Z}^\star\|_F$ (↓)	7.598 (5.908)	7.107 (5.375)	6.803 (3.088)
$\|\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}^\star\|_2$(↓)	31.096 (0.575)	30.970 (0.521)	30.834 (0.568)

Table 3.9: Mean (standard deviation) of evaluation metrics across 15 Monte Carlo replications of data generation with teacher DeepHalo encoder.

e Sparsity Assumption in Credit Card Offer Setting

In merchant-offer data, the most natural ξ_{jt} analog is an unobserved offer–market attractiveness component (placement, targeting intensity, merchant-specific demand) that varies across (merchant, week, segment) and is only partially captured by observed features. Modeling it as sparse deviations η_{jt} is plausible if, within a market, only a subset of offers receive special promotion/placement, while many offers share a common shock.

Therefore, the sparsity assumption in the credit card offer problem can be a justified modeling choice, but only under a clear economic logic: within a given "market" (e.g., month × consumer segment × channel), most offers face essentially the same unobserved environment, while only a small subset of offers receives additional, offer-specific unobserved shocks (targeting, campaigns, hidden features), so the vector of offer-level shocks is sparse around a common market component. Formally, Lu and Shimizu [2025] assume a decomposition

$$\xi_{jt} = \bar{\xi}_t + \eta_{jt}, \qquad \text{with } \eta_{jt} \text{ sparse in } j \text{ for many } t,$$

meaning that for each market t, many products satisfy $\eta_{jt} = 0$ (or are very close to zero), i.e., they share the same market-specific shock. The logical force of this assumption is that it reduces the effective dimension of the unobservables: instead of treating the entire J_t-vector $\boldsymbol{\xi}_t = (\xi_{1t}, \ldots, \xi_{J_t t})$ as free, sparsity implies that only a small number of coordinates deviate from the market baseline, so the data can regulate the remaining degrees of freedom without relying entirely on strong IV restrictions.

Why this is plausible in credit card offers. Consider a standard utility index for consumer i choosing offer j in market t:

$$u_{ijt} = \mathbf{x}_{jt}^\top \boldsymbol{\beta}_i + \alpha_i y_{jt} + \xi_{jt} + \varepsilon_{ijt},$$

where y_{jt} is an endogenous offer term (APR, fee, bonus, or an "effective discount"). In credit card marketing, a natural mechanism is that only a handful of offers in each t are subject to special, unobserved interventions such as sending offers to a selected list of customers or promoting an offer through a specific channel (e.g., email, app, or branch). A stylized representation is

$$\xi_{jt} = \bar{\xi}_t + a_t \cdot \mathbf{1}\{j \in \mathcal{K}_t\} + \varepsilon_{jt}, \qquad |\mathcal{K}_t| \ll J_t,$$

so that $\eta_{jt} = a_t \mathbf{1}\{j \in \mathcal{K}_t\} + \varepsilon_{jt}$ is sparse (only offers in the small set $\mathcal{K}_t$ get the extra shift). Under this logic, sparsity is not merely a statistical convenience: it encodes the operational constraint that firms run a limited number of distinct campaigns/targeting rules per period/segment, so most offers move together with the common environment $\bar{\xi}_t$, while only a few offers get additional idiosyncratic shocks. This is exactly the kind of structure that Lu and Shimizu [2025] exploit to identify and estimate the model when unrestricted $\{\xi_{jt}\}$ would otherwise be too high-dimensional.

When the logic breaks. If issuers continuously personalize many offers each period so that essentially every j has its own nontrivial unobserved component (no small $\mathcal{K}_t$), then η_{jt} is not sparse and the Lu and Shimizu [2025] restriction becomes less credible. In that case, a more appropriate competing structure is low-rank (interactive fixed effects),

$$\xi_{jt} = \boldsymbol{\lambda}_j^\top \mathbf{f}_t + \varepsilon_{jt}, \qquad L \ll \min(J, T),$$

which says there are a few latent common shocks $\mathbf{f}_t$ (such as macroeconomic credit conditions, advertising campaigns, or regulatory changes) with heterogeneous offer-specific exposures $\boldsymbol{\lambda}_j$ rather than only a few offers are special. This is the canonical interactive fixed effects model in Bai [2009]. Thus, the correct logically grounded answer is: the sparsity assumption in [Lu and Shimizu, 2025] can apply to credit card offers when offer-specific unobservables are driven by rare, targeted interventions within each market/time cell. If unobservables are instead pervasive but factor-driven, then the low-rank benchmark of Bai [2009] is the better structural alternative.

Bibliography

Ali Aouad and Antoine Désir. Representing random utility choice models with neural networks, 2023. URL https://arxiv.org/abs/2207.12877.

Vincent Auriau, Ali Aouad, Antoine Désir, and Emmanuel Malherbe. Choice-learn: Large-scale choice modeling for operational contexts through the lens of machine learning. *Journal of Open Source Software*, 9(101):6899, 2024. doi: 10.21105/joss.06899. URL https://doi.org/10.21105/joss.06899.

Jushan Bai. Panel data models with interactive fixed effects. *Econometrica*, 77(4): 1229–1279, 2009. ISSN 00129682, 14680262. URL http://www.jstor.org/stable/40263859.

Steven Berry, James Levinsohn, and Ariel Pakes. Automobile prices in market equilibrium. *Econometrica*, 63(4):841–890, July 1995. doi: None. URL https://ideas.repec.org/a/ecm/emetrp/v63y1995i4p841-90.html.

Michel Bierlaire. Acceptance of modal innovation: the case of the swissmetro. 2001. URL https://api.semanticscholar.org/CorpusID:15826408.

H. D. Block. *Random Orderings and Stochastic Theories of Responses (1960)*, pages 172–217. Springer Netherlands, Dordrecht, 1974. ISBN 978-94-010-9276-0. doi: 10.1007/978-94-010-9276-0_8. URL https://doi.org/10.1007/978-94-010-9276-0_8.

Christopher V. Forinash and Frank S. Koppelman. Application and interpretation of nested logit models of intercity mode choice. *Transportation Research Record*, pages 98–106, 1993. URL https://api.semanticscholar.org/CorpusID:152304413.

William H. Greene and David A. Hensher. A latent class model for discrete choice analysis: contrasts with mixed logit. *Transportation Research Part B: Methodological*, 37(8):681–698, 2003. ISSN 0191-2615. doi: https://doi.org/10.1016/S0191-2615(02)00046-2. URL https://www.sciencedirect.com/science/article/pii/S0191261502000462.

Yafei Han, Francisco Camara Pereira, Moshe Ben-Akiva, and Christopher Zegras. A neural-embedded choice model: Tastenet-mnl modeling taste heterogeneity with flexibility and interpretability, 2022. URL https://arxiv.org/abs/2002.00922.

Stephane Hess, Andrew Daly, and Richard Batley. Revisiting consistency with random utility maximisation: theory and implications for practical work. *Theory and Decision*, 84(2):181–204, March 2018. doi: 10.1007/s11238-017-9651-7. URL https://ideas.repec.org/a/kap/theord/v84y2018i2d10.1007_s11238-017-9651-7.html.

Wagner A. Kamakura and Gary J. Russell. A probabilistic choice model for market segmentation and elasticity structure. *Journal of Marketing Research*, 26(4):379–390, 1989. doi: 10.1177/002224378902600401. URL https://doi.org/10.1177/002224378902600401.

Joohwan Ko and Andrew A. Li. Modeling choice via self-attention, 2024. URL https://arxiv.org/abs/2311.07607.

Zhentong Lu and Kenichi Shimizu. Estimating discrete choice demand models with sparse market-product shocks, 2025. URL https://arxiv.org/abs/2501.02381.

Reza Yousefi Maragheh, Alexandra Chronopoulou, and James Mario Davis. A customer choice model with halo effect. *arXiv: Applications*, 2018. URL https://api.semanticscholar.org/CorpusID:88517466.

Daniel McFadden. Conditional logit analysis of qualitative choice behavior. In Paul Zarembka, editor, *Fontiers in Econometrics*, pages 105–142. Academic press, New York, 1974.

Daniel McFadden. Modelling the choice of residential location. (477), 1978. URL https://EconPapers.repec.org/RePEc:cwl:cwldpp:477.

Amil Petrin and Kenneth Train. A control function approach to endogeneity in consumer choice models. *Journal of Marketing Research*, 47(1):3–13, 2010. doi: 10.1509/jmkr.47.1.3. URL https://doi.org/10.1509/jmkr.47.1.3.

Karlson Pfannschmidt, Pritha Gupta, Björn Haddenhorst, and Eyke Hüllermeier. Learning context-dependent choice functions. *International Journal of Approximate Reasoning*, 140:116–155, 2022. ISSN 0888-613X. doi: https://doi.org/10.1016/j.ijar.2021.10.002. URL https://www.sciencedirect.com/science/article/pii/S0888613X21001614.

Jasper Püschel, Lukas Barthelmes, Martin Kagerbauer, and Peter Vortisch. Comparison of discrete choice and machine learning models for simultaneous modeling of mobility tool ownership in agent-based travel demand models. *Transportation Research Record*,

2678(7):376–390, 2024. doi: 10.1177/03611981231206175. URL https://doi.org/10.1177/03611981231206175.

Brian Sifringer, Virginie Lurkin, and Alexandre Alahi. Enhancing discrete choice models with representation learning. *Transportation Research Part B: Methodological*, 140: 236–261, October 2020. ISSN 0191-2615. doi: 10.1016/j.trb.2020.08.006. URL http://dx.doi.org/10.1016/j.trb.2020.08.006.

Kenneth E. Train. *Discrete Choice Methods with Simulation*. Cambridge University Press, 2nd edition, 2009. Chapter "Logit".

Kenneth E. Train, Daniel McFadden, and Moshe E. Ben-Akiva. The demand for local telephone service: a fully discrete model of residential calling patterns and service choices. *The RAND Journal of Economics*, 18:109–123, 1987. URL https://api.semanticscholar.org/CorpusID:153685064.

Shenhao Wang, Baichuan Mo, Yunhan Zheng, Stephane Hess, and Jinhua Zhao. Comparing hundreds of machine learning and discrete choice models for travel demand modeling: An empirical benchmark. *Transportation Research Part B: Methodological*, 190:103061, December 2024. ISSN 0191-2615. doi: 10.1016/j.trb.2024.103061. URL http://dx.doi.org/10.1016/j.trb.2024.103061.

Melvin Wong and Bilal Farooq. Reslogit: A residual neural network logit model for data-driven choice modelling. *Transportation Research Part C: Emerging Technologies*, 126: 103050, 2021. ISSN 0968-090X. doi: https://doi.org/10.1016/j.trc.2021.103050. URL https://www.sciencedirect.com/science/article/pii/S0968090X21000802.

Yiqi Yang, Zhi Wang, Rui Gao, and Shuang Li. Reproducing kernel hilbert space choice model. In *Proceedings of the 26th ACM Conference on Economics and Computation*, EC '25, page 819, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400719431. doi: 10.1145/3736252.3742630. URL https://doi.org/10.1145/3736252.3742630.

Shuhan Zhang, Zhi Wang, Rui Gao, and Shuang Li. Deep context-dependent choice model. In *2nd Workshop on Models of Human Feedback for AI Alignment*, 2025. URL https://openreview.net/forum?id=bXTBtUjb0c.